DB Learning Series

Amazon DynamoDB

Complete Technical Knowledge Base

From First Principles to Production Operations — Architecture, Security, Performance, HA/DR, Troubleshooting & Interview Prep

Author: Praveen.M (PMSQLDBA)

July 2026

Audience: DBAs, Cloud/Data Engineers, Solution Architects, SRE

Scope: DynamoDB core service, Streams, DAX, Global Tables, PITR, AWS Backup

Table of Contents

1. Overview 5
1.1 What Is Amazon DynamoDB? 5
1.2 Core Characteristics 5
1.3 DynamoDB vs. Relational (SQL Server) — Translation Table 6
1.4 Capacity Modes 6
1.5 Cost Model 7
1.6 When NOT to Use DynamoDB 8
2. Architecture 9
2.1 The 10,000-Foot View 9
2.2 Partitioning — The Concept That Governs Everything 10
2.3 Partition Splits 11
2.4 Replication and Consistency 11
2.5 Data Plane vs. Control Plane 12
3. Components 14
3.1 Primary Keys 14
3.2 Attribute Data Types 15
3.3 Secondary Indexes — GSI vs. LSI 15
3.4 DynamoDB Streams 17
3.5 TTL (Time To Live) 17
3.6 DAX (DynamoDB Accelerator) 18
3.7 Global Tables 19
4. Installation 20
4.1 Install the AWS CLI v2 20
4.2 DynamoDB Local (Docker) — the developer's best friend 20
4.3 Install the SDKs 21
4.4 Create Your First Table — Three Ways 22
5. Configuration 25
5.1 Credential Configuration 25
5.2 Client-Side Configuration That Actually Moves the Needle 25
5.3 Table-Level Configuration Reference 26
5.4 VPC Endpoint Configuration 28
6. Prerequisites 29
6.1 Account & Platform Prerequisites 29
6.2 The Design Prerequisite — Access Pattern Enumeration 29
6.3 Service Quotas Worth Knowing 30
6.4 Skills Prerequisites for the Team 31

7. Security Best Practices 32
7.1 IAM — Least Privilege in Practice 32
7.2 Encryption 34
7.3 Auditing & Compliance 35
7.4 Security Checklist 36
8. Performance Tuning 37
8.1 Capacity Unit Arithmetic — learn this cold 37
8.2 Hot Partitions — the #1 production performance problem 38
8.3 Query vs. Scan 39
8.4 Item Size Optimisation 40
8.5 Batching, Parallelism, and Transactions 41
8.6 Performance Tuning Checklist 42
9. Monitoring 44
9.1 Essential CloudWatch Metrics 44
9.2 Alarms Worth Paging For 45
9.3 Contributor Insights — finding the hot key 46
9.4 The Dashboard 46
10. Logging 48
10.1 CloudTrail — Management vs. Data Events 48
10.2 Building an Immutable Audit Log with Streams 49
11. Backup & Recovery 51
11.1 Point-In-Time Recovery 51
11.2 AWS Backup + Vault Lock — the ransomware control 53
11.3 S3 Export — backup for analytics 54
12. Disaster Recovery 55
12.1 Match the Strategy to the Disaster 55
12.2 Global Tables — the operational reality 56
12.3 DR Runbook Template 56
12.4 Game Days 57
13. High Availability 58
13.1 What AWS Gives You for Free 58
13.2 HA Comparison 58
13.3 The HA You Do Have To Build 59
14. Troubleshooting 61
14.1 Incident 1 — Throttling at 5% Table Utilisation 61
14.2 Incident 2 — GSI Throttling Stalls the Base Table 62
14.3 Incident 3 — 'It's Slow' but the Server Is Fast 62
14.4 Incident 4 — TTL Items Never Expire 63

14.5 Incident 5 — Stream Consumer Lag Approaching 24 Hours 64
14.6 Triage Command Cheat Sheet 65
15. Common Errors 66
15.1 The Reserved-Word Trap 67
15.2 The `TransactionCanceledException` Puzzle 67
15.3 Silent Failures — the dangerous category 68
16. FAQs 70
17. Interview Questions 73
17.1 Level 1 — Screening 73
17.2 Level 2 — Mid-Level 73
17.3 Level 3 — Senior 74
17.4 Level 4 — Architect / Scenario 75
18. Real-World Use Cases 77
18.1 Session Store 77
18.2 IoT Time-Series 77
18.3 Shopping Cart (the original Dynamo use case) 77
18.4 Gaming Leaderboard 78
18.5 Financial Ledger (compliance-heavy) 78
18.6 Event Sourcing / CQRS 79
19. Best Practices 80
19.1 Data Modelling 80
19.2 Operations 80
19.3 Application Code 80
19.4 Security 81
20. References 82
20.1 AWS Primary Documentation 82
20.2 Foundational Reading 83
20.3 Certification Alignment 83
20.4 Document Control ..... **Error! Bookmark not defined.**

1. Overview

What DynamoDB is, what problem it solves, and — just as important — when you should not use it.

1.1 What Is Amazon DynamoDB?

Amazon DynamoDB is a fully managed, serverless, multi-region, key-value and document NoSQL database delivered as an AWS service. There is no server to patch, no instance to size, no storage volume to grow, and no failover cluster to build. You create a **table**, define a **primary key**, and issue API calls over HTTPS. AWS transparently partitions your data across many storage nodes in three Availability Zones (AZs) and scales throughput and storage independently of one another.

DynamoDB descends directly from the 2007 Amazon **Dynamo** paper, which introduced consistent hashing, quorum replication, and eventual consistency to solve Amazon's shopping-cart availability problem. The commercial service launched in 2012 and swapped Dynamo's client-side quorum model for a managed, leader-based replication design with an internal Paxos control plane.

> **NOTE — The single most important mental model**
> A relational engine like SQL Server optimises **flexible queries over normalised data**. DynamoDB optimises **predictable latency at unbounded scale for known access patterns**. In SQL Server you design the schema first and write queries later. In DynamoDB you enumerate every access pattern first and design the key schema to serve them. Getting this backwards is the root cause of roughly 90% of failed DynamoDB projects.

1.2 Core Characteristics

Table 1.1 — DynamoDB at a glance

Characteristic	What it means in practice	Hard limit / detail
Fully managed	No OS, no patching, no VM, no storage admin. Control plane is an API.	AWS-managed; no maintenance windows
Serverless	Provisioned or On-Demand capacity; you never see a node.	Scales to zero cost in On-Demand
Single-digit ms latency	Consistent P50 ~2–5 ms for point reads at any table size.	DAX reduces reads to microseconds
Durability	Every write is synchronously replicated to 3 AZs before ACK.	3-way replicated, quorum 2/3
Availability SLA	99.99% single-Region; 99.999% with Global Tables.	Financially backed SLA
Elastic scale	Tables grow to petabytes and millions of req/sec.	No table size limit
Item size	Each item is a self-contained document.	**400 KB max** (item + attribute names)
Consistency	Eventually consistent by default; strongly consistent option per read.	Strong reads cost 2× and are AZ-local
Transactions	ACID across up to 100 items / 4 MB.	TransactWriteItems / TransactGetItems

Characteristic	What it means in practice	Hard limit / detail
Query model	Key-based only. No joins, no ad-hoc WHERE on arbitrary columns.	PartiQL provides SQL-like syntax, not SQL semantics

1.3 DynamoDB vs. Relational (SQL Server) — Translation Table

Table 1.2 — Vocabulary mapping for the relational DBA

SQL Server concept	DynamoDB equivalent	Critical difference
Database / instance	AWS account + Region	No instance object exists at all
Table	Table	DynamoDB tables are schemaless except for the key
Row	Item	Items in one table may have entirely different attributes
Column	Attribute	Attributes are per-item, not per-table
Primary key / clustered index	Partition key (+ optional sort key)	Determines physical placement; cannot be changed later
Non-clustered index	GSI (Global Secondary Index)	GSI is a **separate replicated table** with its own capacity
Filtered / covering index	LSI (Local Secondary Index)	Only creatable at table creation; shares partition key
JOIN	— (none)	You denormalise or perform multiple round trips
Transaction log / CDC	DynamoDB Streams / Kinesis Data Streams	24-hour retention (Streams); 365 days (KDS)
Always On AG	Global Tables (multi-Region, active-active)	Last-writer-wins, not synchronous commit
Backup / RESTORE	On-demand backup + PITR	PITR restores to a **new table**, never in place
Query optimiser statistics	— (none)	Cost is deterministic; there is no plan to tune
Buffer pool	DAX (opt-in write-through cache)	Not automatic; you provision a DAX cluster

1.4 Capacity Modes

Every table runs in exactly one of two capacity modes, switchable once every 24 hours.

Table 1.3 — On-Demand vs. Provisioned

Dimension	On-Demand (Pay-per-request)	Provisioned (RCU/WCU)
Billing unit	Per request (RRU / WRU)	Per capacity-unit-hour, reserved
Cost per unit	~6–7× the provisioned rate	Cheapest at steady, high utilisation

Dimension	On-Demand (Pay-per-request)	Provisioned (RCU/WCU)
Scaling	Instant to 2× previous peak; then adapts in ~30 min	Manual, or Application Auto Scaling (minutes)
Throttling risk	Low, but real when traffic >2× prior peak	High if scaling lags a spike
Best for	Spiky, new, unknown or dev/test workloads	Predictable, steady, high-volume workloads
Reserved Capacity	Not available	1- or 3-year RC gives up to ~76% discount
Break-even rule of thumb	Choose On-Demand below ~14–18% average utilisation	Choose Provisioned above ~20% sustained utilisation

TIP — Practical capacity strategy

Launch new tables in **On-Demand**, collect two to four weeks of `ConsumedReadCapacityUnits` / `ConsumedWriteCapacityUnits` from CloudWatch, then decide. If the P95/P50 ratio is under ~3 and utilisation is steady, move to Provisioned + Auto Scaling + Reserved Capacity. On-Demand now supports a **maximum throughput cap**, which is the correct guardrail against a runaway loop billing you thousands of dollars overnight.

1.5 Cost Model

Table 1.4 — What you actually pay for (us-east-1, indicative)

Billing dimension	Indicative price	Engineering implication
Provisioned WCU	~$0.00065 / WCU-hour	1 WCU = 1 write/sec of ≤1 KB
Provisioned RCU	~$0.00013 / RCU-hour	1 RCU = 1 strong read/sec of ≤4 KB, or 2 eventual reads
On-Demand WRU	~$1.25 / million writes	Same 1 KB rounding rule
On-Demand RRU	~$0.25 / million strong reads	Eventually consistent reads are half price
Storage	~$0.25 / GB-month	Includes ~100 bytes/item overhead; GSIs pay again
Streams	~$0.02 / 100k read requests	First shard-level reads via Lambda are free
Global Tables	Replicated WRUs (~1.5× WCU)	Cross-Region data transfer is charged separately
Backup	~$0.10 / GB-month (on-demand); ~$0.20 (PITR)	PITR is billed on table size, continuously
DAX	Per node-hour (EC2-like)	Not serverless; you size the cluster

WARNING — The three costs that surprise everyone

- **GSIs double (or triple) your write bill.** Every write to a base table item that contains a GSI key writes to that GSI as well, at full WCU cost. A table with three GSIs can cost 4× to write.
- **`Scan` reads and bills every item, even filtered ones.** `FilterExpression` is applied *after* the read. Filtering 1,000,000 items down to 10 still bills for 1,000,000 items.
- **Storage of a hot 400 KB item.** Reads round *up* to 4 KB blocks — a 400 KB item costs 100 RCU per strong read. One such read per second costs more than most small tables in total.

1.6 When NOT to Use DynamoDB

Seniority shows in knowing the anti-patterns. Reach for another engine when you see any of the following:

- **Ad-hoc analytics / BI.** Unknown, changing query shapes. Use Athena, Redshift, or export to S3.
- **Relational joins across many entities** with low-latency requirements and unpredictable access paths. Use Aurora / RDS.
- **Items larger than 400 KB**, or large blobs. Store the object in S3 and keep only the pointer in DynamoDB.
- **Aggregations** — `SUM`, `COUNT(*)`, `GROUP BY` over large sets. Maintain counters via Streams, or offload to Athena.
- **Full-text search / fuzzy matching.** Use OpenSearch fed from Streams.
- **Complex multi-row referential integrity** with foreign keys and cascading deletes. DynamoDB has no FKs.
- **Very low volume with rich query needs.** A small Aurora Serverless v2 instance is often cheaper and far more flexible.

> **INTERVIEW ANGLE — How to answer 'Why DynamoDB?' in an interview**
> Never say "because it's fast and scales." Say: *"Because the access patterns are known, bounded and key-based; we need predictable single-digit-millisecond latency at a scale where a relational engine would need sharding; and we want zero operational overhead. If those three things were not true — particularly the first — I would push back and recommend Aurora."* That answer demonstrates you understand the trade-off, not just the marketing.

2. Architecture

How the service actually works underneath the API: request routing, partitioning, replication, and the internal control plane.

2.1 The 10,000-Foot View

A DynamoDB request never touches a database instance you own. It hits a Region-wide HTTPS endpoint, is authenticated by AWS Signature Version 4, authorised by IAM, routed by a fleet of stateless request routers to the correct storage partition, and served by that partition's leader (for writes and strongly consistent reads) or any replica (for eventual reads).

MERMAID DIAGRAM — SERVICE TOPOLOGY — WHAT SITS BETWEEN YOUR CODE AND YOUR DATA

```
flowchart TB
    subgraph Client["Your environment"]
        APP["Application / AWS SDK"]
        DAX["DAX cluster — optional, runs in YOUR VPC<br/>the only non-serverless component"]
    end

    subgraph Edge["AWS front door — Region-wide, stateless, AWS-owned"]
        EP["Regional endpoint — a DNS name, not a host<br/>dynamodb.us-east-1.amazonaws.com"]
        RR["Request router fleet<br/>· verifies the SigV4 signature<br/>· authorizes — IAM + SCP +
endpoint/resource policy + KMS<br/>· resolves the partition from its CACHED map"]
        MET["Metadata service<br/>authoritative partition map"]
    end

    subgraph Storage["Storage layer — one Region, three AZs"]
        subgraph RG1["Partition 1 — replication group"]
            P1L["Leader (AZ-a)"]
            P1R1["Replica (AZ-b)"]
            P1R2["Replica (AZ-c)"]
        end
        RG2["Partition 2 — replication group<br/>leader + 2 replicas across AZ-a/b/c"]
        RGN["... Partition N — same shape, always"]
    end

    subgraph Control["Control plane — NEVER in the request path"]
        AS["Auto admin<br/>split for size · split for heat · node repair"]
        PAX["Paxos — group membership and leader election<br/>for EVERY replication group, not
just one"]
    end

    APP --- DAX
    DAX --- EP
    APP --- EP
    EP --- RR
    RR -->|"read on cache miss only — NOT per request"| MET
    RR --- RG1
    RR --- RG2
    RR --- RGN
    P1L -->|"replicates to"| P1R1
    P1L --> P1R2
    AS -.- RG1
    AS -.- RG2
    PAX -.- RG1
    PAX -.- RG2
```

Figure 2.1 — Topology, not sequence: solid undirected links mean "can reach", so a router reaching every partition is a statement of connectivity, not of one request visiting all of them. Figure 2.3 owns the ordered request path. Note what is NOT here: any host you own, size or patch. The router caches the partition map — a metadata lookup per request would make the single-digit-millisecond latency in §1.2 impossible.

2.2 Partitioning — The Concept That Governs Everything

DynamoDB hashes the **partition key** with an internal hash function. The hash output maps onto a keyspace, and each contiguous range of that keyspace is owned by exactly one partition. A partition is the unit of storage and throughput. (The 2007 Dynamo paper used a consistent-hashing *ring* with client-side routing; the managed service instead tracks range ownership in the metadata service — the vocabulary survives, the mechanism does not.)

Table 2.1 — Partition physics

Property	Limit	Consequence
Storage per partition	10 GB	Exceeding it triggers an automatic split
Write throughput per partition	1,000 WCU/sec	A single partition key cannot absorb more (except via write sharding)
Read throughput per partition	3,000 RCU/sec	The famous '3,000 RCU / 1,000 WCU per partition' rule
Item collection (partition + all LSIs)	10 GB	**LSI-only limit** — the single hardest DynamoDB failure to fix later
Replicas per partition	3 (one per AZ)	Synchronous quorum write of 2/3 before ACK

MERMAID DIAGRAM — HASH-BASED PARTITION PLACEMENT

```
flowchart LR
    subgraph Items["Incoming write requests"]
        A["PK = CUST#1001"]
        B["PK = CUST#1002"]
        C["PK = CUST#9007"]
    end

    H{{"hash(PK)"}}

    subgraph Ring["Keyspace — one contiguous range per partition"]
        P1["Partition 1<br/>0x00–0x55"]
        P2["Partition 2<br/>0x56–0xAB"]
        P3["Partition 3<br/>0xAC–0xFF"]
    end

    A --> H
    B --> H
    C --> H

    H -->|"CUST#1001 → 0x1F"| P1
    H -->|"CUST#1002 → 0x7A"| P2
    H -->|"CUST#9007 → 0xE3"| P3

    P1 --- R1["Replication group<br/>leader + 2 replicas · AZ-a/b/c<br/>≤10 GB · 3,000 RCU · 1,000 WCU"]
    P2 --- R2["Replication group<br/>leader + 2 replicas · AZ-a/b/c<br/>≤10 GB · 3,000 RCU · 1,000 WCU"]
    P3 --- R3["Replication group<br/>leader + 2 replicas · AZ-a/b/c<br/>≤10 GB · 3,000 RCU · 1,000 WCU"]
```

Figure 2.2 — Each key hashes to one value, that value falls in exactly one range, and that range is one partition. Every partition carries the same ceiling. If all three keys hashed into 0x00–0x55, P1 would absorb everything while P2 and P3 sat idle — that is a hot partition.

2.3 Partition Splits

Partitions split automatically, and the mechanism matters because splits are **one-way**: DynamoDB never merges partitions back. If you provision 100,000 WCU for a one-off migration, the table splits into ~100 partitions, and each one keeps only ~1/100th of your throughput after you scale back down.

Table 2.2 — Split triggers

Trigger	Mechanism	Reversible?
Size (>10 GB)	Split by key range at the midpoint	No
Throughput (provisioned increase)	Split for capacity — partition count = f(peak provisioned)	No — the classic 'throughput dilution' trap
Heat (adaptive capacity)	Split for heat — isolates a hot key range onto its own partition	No; automatic and free since 2019

WARNING — Throughput dilution — a real production incident pattern

Table starts at 1,000 WCU (1 partition). A bulk load provisions 100,000 WCU → DynamoDB splits into ~100 partitions. You scale back to 1,000 WCU. Each partition now gets ~10 WCU. Your steady-state workload, which never had a problem before, now throttles at a fraction of its old volume until adaptive capacity redistributes heat. **Mitigation:** for one-off loads, use a separate staging table, or scale up gradually, or use On-Demand.

2.4 Replication and Consistency

Each partition is a replication group of three nodes across three AZs. One node is the leader, elected via an internal Paxos protocol. Writes go to the leader, which persists to its own write-ahead log and forwards to both replicas; the leader ACKs the client once **two of three** (itself + one replica) have durably written. This is why a write is durable across at least two AZs before your SDK returns.

MERMAID DIAGRAM — WRITE PATH AND CONSISTENCY MODELS

```
sequenceDiagram
    participant C as Client SDK
    participant R as Request Router
    participant L as Leader (AZ-a)
    participant R1 as Replica (AZ-b)
    participant R2 as Replica (AZ-c)

    C->>R: PutItem (SigV4 signed)
    R->>L: route by hash(PK)
    L->>L: append to write-ahead log
    par Leader replicates to both AZs
        L->>R1: replicate
    and
        L->>R2: replicate
    end
    R1-->>L: ACK
    Note over L: quorum reached — leader + 1 replica = 2 of 3
    L-->>R: committed
    R-->>C: 200 OK  (durable in 2+ AZs)
    R2-->>L: ACK (lags slightly)
```

```
    Note over C,R2: Strongly consistent read — always served by the Leader
    Note over C,R2: Eventually consistent read — any replica. A read served by R2<br/>inside the
window above may miss the newest write<br/>(typically under 1s, but not SLA-bounded)
```

Figure 2.3 — Quorum write. The leader counts itself, so one replica ACK is enough to commit. An eventually consistent read routed to R2 before its ACK returns stale data — that window is normally sub-second but is not bounded by any SLA.

Table 2.3 — Read consistency options

Model	Cost	Latency	When to use
Eventually consistent	0.5 RCU per 4 KB	Lowest	**Default.** Product catalogues, feeds, dashboards, anything tolerant of ~1s staleness
Strongly consistent	1 RCU per 4 KB	Slightly higher; leader-only	Read-after-write correctness: balances, inventory counts, idempotency checks
Transactional read	2 RCU per 4 KB	Highest	Snapshot isolation across multiple items

CRITICAL — Two consistency traps

- **GSIs are ALWAYS eventually consistent.** `ConsistentRead=true` on a GSI Query throws `ValidationException`. If your code needs read-after-write on an index, redesign — do not argue with the API.
- **Global Tables are eventually consistent across Regions and use last-writer-wins.** Two concurrent writes to the same item in two Regions: one silently disappears. Never run a bank ledger active-active on Global Tables without an idempotency/conflict layer.

2.5 Data Plane vs. Control Plane

Table 2.4 — Two very different APIs

Aspect	Data plane	Control plane
Operations	GetItem, PutItem, Query, Scan, UpdateItem, BatchWriteItem, TransactWriteItems	CreateTable, UpdateTable, DeleteTable, CreateBackup, UpdateContinuousBackups
Latency	Single-digit ms	Seconds to minutes (async, eventually consistent)
Consistency	Strong option available	Eventually consistent — `DescribeTable` may lag reality
Throttling	Capacity-based (`ProvisionedThroughputExceeded`)	API rate limits (`LimitExceededException`)
Logged in CloudTrail	Only if data-event logging is explicitly enabled (chargeable)	Always, free, in the management event trail
Blast radius of an IAM mistake	Data exposure	**Table deletion** — always deny `dynamodb:DeleteTable` broadly

INTERVIEW ANGLE — Interview angle

> "Walk me through what happens when I call PutItem." The complete answer: SDK signs with SigV4 → regional endpoint → IAM authorisation (including any condition keys such as `dynamodb:LeadingKeys`) → request router looks up the partition map in the metadata service → routes to the leader of the partition owning `hash(PK)` → leader writes WAL and replicates → ACK at quorum 2/3 → optionally the change is emitted to Streams. Mentioning the quorum and the metadata service is what separates a senior answer from a junior one.

3. Components

Every object you will create, name, secure, monitor and pay for.

MERMAID DIAGRAM — COMPONENT MAP — WHAT A TABLE IS MADE OF

```
flowchart TB
    T["DynamoDB TABLE"]

    T --- PK["Primary Key — immutable, chosen once and forever"]
    PK --- SPK["Simple — Partition Key only"]
    PK --- CPK["Composite — Partition Key + Sort Key"]

    T --- IT["Items — ≤400 KB each, schemaless beyond the key"]
    IT --- AT["Attributes<br/>S · N · B · BOOL · NULL · L · M · SS · NS · BS"]

    T --- IDX["Secondary Indexes"]
    IDX --- GSI["GSI — any PK/SK · own capacity · eventual reads only<br/>20 per table · add or
drop online, anytime"]
    IDX --- LSI["LSI — same PK, alt SK · shares base capacity · strong reads OK<br/>5 per table ·
create-time only, undroppable · 10 GB cliff"]

    T --- CAP["Capacity Mode — one at a time, switch once per 24h"]
    CAP --- OD["On-Demand — billed per request"]
    CAP --- PR["Provisioned + Auto Scaling — billed per capacity-hour"]

    T --- CDC["Change Data Capture — either, or both"]
    CDC --- SD["DynamoDB Streams — 24h retention"]
    CDC --- KDS["Kinesis Data Streams — up to 365d retention"]
    SD --- LAM["Lambda / KCL consumers"]
    KDS --- LAM

    T --- TTL["TTL — free background expiry, ~48h best-effort"]
    T --- BK["Backup — PITR 35d · on-demand · AWS Backup + Vault Lock"]
    T --- GT["Global Tables — multi-Region active-active, last-writer-wins"]
    T --- ENC["Encryption at rest — ALWAYS on<br/>AWS-owned · AWS-managed · customer-managed CMK"]
    T --- DAXC["DAX — in-memory cache cluster, opt-in"]
```

Figure 3.1 — Links here mean "has a", not "leads to". This is an inventory of what a table is composed of, deliberately drawn with undirected links so that no edge is mistaken for a flow.

3.1 Primary Keys

Table 3.1 — Key schema types

Type	Structure	Access capability
Simple key	Partition key (HASH) only	`GetItem` by exact key. Values must be unique. Pure key-value.
Composite key	Partition key (HASH) + Sort key (RANGE)	`GetItem` by exact pair, **plus** `Query` on PK with `=`, `<`, `>`, `BETWEEN`, `begins_with` on SK. This is where all the design power lives.

NOTE — Key constraints worth memorising

- Partition key: max **2048 bytes**. Sort key: max **1024 bytes**.
- Key attributes must be scalar — `String`, `Number` or `Binary` only. No Boolean, no List, no Map.
- Key schema is **immutable**. Changing it means creating a new table and migrating.

- `Query` **requires** an equality condition on the partition key. There is no way to Query across partition keys — that is a `Scan`.

3.2 Attribute Data Types

Table 3.2 — DynamoDB data types

Type	Code	Description	DBA notes
String	S	UTF-8, up to 400 KB	Empty strings allowed in non-key attributes since 2020
Number	N	38 digits of precision	Transmitted as a string; SDKs may lose precision — use `Decimal` in Python
Binary	B	Base64-encoded on the wire	Base64 inflates size ~33% against the 400 KB limit
Boolean	BOOL	true / false	Cannot be a key attribute
Null	NULL	Explicit null marker	Different from 'attribute absent' — sparse-index behaviour differs
List	L	Ordered, heterogeneous	Update individual elements by index: `SET tags[2] = :v`
Map	M	Nested key/value document	The JSON-document type; 32 levels of nesting max
String Set	SS	Unordered, unique strings	`ADD` / `DELETE` are atomic — great for tag sets
Number Set	NS	Unordered, unique numbers	Sets cannot be empty; delete the attribute instead
Binary Set	BS	Unordered, unique binaries	Rare

3.3 Secondary Indexes — GSI vs. LSI

Table 3.3 — The comparison you will be asked about in every interview

Dimension	GSI (Global Secondary Index)	LSI (Local Secondary Index)
Partition key	**Any** attribute	**Must** be the same as the base table's PK
Sort key	Any attribute, optional	Any attribute, **required**
Create/delete time	Any time, online	**Table creation only** — never after
Delete after creation	Yes	**No** — requires table rebuild
Capacity	**Separate** RCU/WCU from the base table	Shares the base table's capacity
Consistency	**Eventually consistent only**	Strongly consistent reads supported
Item collection size limit	None	**10 GB per partition key** across base + all LSIs
Max per table	20 (soft limit, raisable)	5 (hard)
Projection	KEYS_ONLY / INCLUDE / ALL	KEYS_ONLY / INCLUDE / ALL

Dimension	GSI (Global Secondary Index)	LSI (Local Secondary Index)
Throttling behaviour	GSI throttling **backpressures base-table writes**	Base capacity shared; contention possible
Cost impact	Full extra WCU per replicated write	Extra WCU only when the LSI attributes change

CRITICAL — The LSI 10 GB cliff

If a table has **any** LSI, the total size of all items sharing one partition key — base table plus every LSI — must stay under 10 GB. Exceed it and every write to that partition key fails with `ItemCollectionSizeLimitExceededException`. There is no fix short of rebuilding the table without the LSI. **Default position: use a GSI unless you provably need strongly consistent index reads.** In 2026 practice, that is almost never.

3.3.1 Projections

Table 3.4 — Index projection types

Projection	Contains	Trade-off
KEYS_ONLY	Index key + base table key	Smallest & cheapest; every read needs a second `GetItem` on the base table
INCLUDE	Keys + a named list of attributes	**Usually correct.** Covering index without paying for everything
ALL	Every attribute of the item	Fully covering, no fetch-back; doubles storage and write cost

3.3.2 Sparse Indexes — the underused superpower

An item only appears in a GSI if it **has** the GSI's key attributes. Omit the attribute and the item is invisible to that index. This turns a GSI into a filtered index for pennies.

```python
# Pattern: an index over ONLY unprocessed orders in a 500-million-item table.
# GSI key: gsi_pk = 'PENDING'  (attribute exists only while status is pending)

# 1. Order created -> attribute present -> item lands in the sparse GSI
table.put_item(Item={
    'PK': 'ORDER#9001', 'SK': 'META',
    'status': 'PENDING',
    'gsi_pk': 'PENDING',            # <-- membership token
    'gsi_sk': '2026-07-15T09:30:00Z'
})

# 2. Worker processes it -> REMOVE the attribute -> item vanishes from the GSI
table.update_item(
    Key={'PK': 'ORDER#9001', 'SK': 'META'},
    UpdateExpression='SET #s = :done REMOVE gsi_pk, gsi_sk',
    ExpressionAttributeNames={'#s': 'status'},
    ExpressionAttributeValues={':done': 'FULFILLED'}
)
```

```
# 3. The queue query now scans ONLY pending work - never the 500M table
resp = table.query(
    IndexName='GSI_PendingQueue',
    KeyConditionExpression=Key('gsi_pk').eq('PENDING'),
    Limit=100
)
```

Listing 3.1 — A sparse GSI as a work queue. The index contains only in-flight items, so its size and cost are proportional to the backlog, not the table.

3.4 DynamoDB Streams

Streams is DynamoDB's change data capture: an ordered, per-partition-key log of every item-level modification, retained for 24 hours, delivered exactly-once to Lambda (or via the KCL adapter).

Table 3.5 — StreamViewType options

StreamViewType	Record contains	Typical use
KEYS_ONLY	Only the key attributes	Cache invalidation, notification triggers
NEW_IMAGE	The item after modification	Replication to OpenSearch / S3
OLD_IMAGE	The item before modification	Audit trail of deletions
NEW_AND_OLD_IMAGES	Both	**Most common.** Audit, deltas, aggregation, CDC to a warehouse

WARNING — Streams facts that bite in production

- **24-hour retention, full stop.** A broken consumer for 25 hours means permanent data loss on the change log. Alarm on `IteratorAge` at 4 hours — not at 23.
- Ordering is guaranteed **per partition key**, not globally.
- Lambda retries a failed batch **until the record expires**, blocking that shard. Always configure `BisectBatchOnFunctionError`, `MaximumRetryAttempts` and an `OnFailure` destination (SQS DLQ).
- **TTL deletions do appear in the Stream**, flagged with `userIdentity.principalId = 'dynamodb.amazonaws.com'`. This is how you archive expired items to S3.

3.5 TTL (Time To Live)

TTL is a free background process that deletes items whose designated attribute holds a **Unix epoch time in seconds** in the past. It consumes no WCU. It is not a scheduler: deletion typically happens within 48 hours of expiry, not at the second.

BASH

```
# Enable TTL on the 'expires_at' attribute
aws dynamodb update-time-to-live \
  --table-name Sessions \
  --time-to-live-specification 'Enabled=true,AttributeName=expires_at'

# Verify
aws dynamodb describe-time-to-live --table-name Sessions

# The attribute MUST be type N and MUST be epoch SECONDS (not ms, not ISO-8601)
#   Correct : "expires_at": {"N": "1784000000"}
```

```
#   WRONG   : "expires_at": {"S": "2026-07-15"}     <- silently ignored forever
#   WRONG   : "expires_at": {"N": "1784000000000"}  <- milliseconds = year 58,500
```

Listing 3.2 — Enabling TTL. The two 'WRONG' lines are the most common production TTL bug: no error is raised, items simply never expire and your storage bill grows forever.

TIP — Filter expired items in the read path

Because deletion is best-effort within 48 hours, expired items remain readable. If correctness depends on expiry (session tokens, feature flags), add `FilterExpression="expires_at > :now"` to your queries. Treat TTL as **cost cleanup**, not as an access-control mechanism.

3.6 DAX (DynamoDB Accelerator)

DAX is a managed, write-through, in-memory cache that speaks the DynamoDB API. Your application changes only the endpoint and the client library; the code stays the same. It turns millisecond reads into microsecond reads.

MERMAID DIAGRAM — DAX TOPOLOGY AND CACHES

```
flowchart LR
    APP["Application (in VPC)"]

    subgraph CL["DAX cluster — 3 nodes, one per AZ<br/>every node holds BOTH caches"]
        PRI["Primary node (AZ-a)<br/>item cache + query cache"]
        RR1["Read replica (AZ-b)<br/>item cache + query cache"]
        RR2["Read replica (AZ-c)<br/>item cache + query cache"]
    end

    DDB[("DynamoDB Table")]

    APP -->|"writes — primary only"| PRI
    APP -->|"reads — any node"| PRI
    APP --> RR1
    APP --> RR2

    PRI -->|"write-through"| DDB
    PRI -.->|"propagate to replicas"| RR1
    PRI -.-> RR2
    RR1 -.->|"cache miss → read"| DDB
    RR2 -.-> DDB

    IC["Item cache — GetItem / BatchGetItem · default TTL 5 min"]
    QC["Query cache — Query / Scan result sets · default TTL 5 min"]
    CL --- IC
    CL --- QC
```

Figure 3.2 — DAX runs inside your VPC on nodes you size — the one non-serverless part of the DynamoDB ecosystem. Both caches live on every node; writes go through the primary, reads can be served by any node.

Table 3.6 — When DAX is and is not the answer

Use DAX when…	Do NOT use DAX when…
Read-heavy, repeated reads of the same items (hot catalogue, config, leaderboards)	Write-heavy workloads — DAX adds latency on the write path
Microsecond latency is a genuine requirement	You need strongly consistent reads (DAX passes them straight through, uncached)
A read hot spot is throttling despite adaptive capacity	The application cannot tolerate up-to-5-minute staleness

Use DAX when…	Do NOT use DAX when…
Read cost dominates your bill and cacheability is high	The table is written outside DAX (direct writes make the cache stale until TTL)

CRITICAL — The DAX cache-coherence trap

DAX is only write-through for writes that go **through DAX**. A Lambda, a Stream consumer, a batch job or an admin using the console that writes directly to the table leaves DAX serving stale data until the item TTL expires. **Rule: if you adopt DAX, every writer for that table must go through DAX — or you must lower the item TTL and accept the staleness explicitly.**

3.7 Global Tables

Global Tables replicate a table to additional Regions with active-active, multi-master semantics. Streams (Version 2019.11.21 onward) carries the replication; conflicts resolve by **last-writer-wins** using an internal timestamp.

Table 3.7 — Global Tables essentials

Property	Detail
Version	Always use **2019.11.21 (Version 2)**. Version 2017.11.29 is legacy and cannot add Regions to a live table cleanly.
Replication lag	Typically **under one second**; not SLA-bounded. Watch `ReplicationLatency`.
Conflict resolution	**Last-writer-wins.** Concurrent writes to one item in two Regions: one is silently discarded.
Prerequisites	Streams enabled with `NEW_AND_OLD_IMAGES`; identical table name in every Region.
Capacity	Each replica has its own capacity. **Provision write capacity in every Region for the *global* write volume**, not the local one.
Availability SLA	99.999% (vs. 99.99% single-Region)
Cost	Replicated writes are billed as rWCU/rWRU (~1.5×) plus cross-Region transfer.
Multi-Region Strong Consistency	Available on newer Global Tables configurations for workloads that cannot tolerate LWW — evaluate cost and latency before adopting.

INTERVIEW ANGLE — Interview angle — GSI vs. LSI

The winning answer is not the feature list; it is the recommendation with a reason: *"I default to GSIs. LSIs must be declared at table creation, can never be dropped, share base capacity, and impose a hard 10 GB item-collection limit per partition key — which is a production landmine. The only thing an LSI buys you is strongly consistent index reads. If a design genuinely requires that, I would first ask whether the data belongs in the base item instead."*

4. Installation

DynamoDB is a service — there is nothing to install. What you install is the tooling, the local emulator, and the SDKs.

> **NOTE — Read this before you look for a setup.exe**
> There is no DynamoDB server to install, no port to open, no service account to create, and no version to patch. "Installation" for DynamoDB means three things: (1) the AWS CLI and SDKs on your workstation and build agents; (2) **DynamoDB Local** for offline development and CI; and (3) creating the table itself, which is an API call, not an install.

4.1 Install the AWS CLI v2

```bash
# --- Linux (x86_64) ---
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o awscliv2.zip
unzip -q awscliv2.zip && sudo ./aws/install --update

# --- Windows (PowerShell, as Administrator) ---
# msiexec.exe /i https://awscli.amazonaws.com/AWSCLIV2.msi /qn

# --- macOS ---
# brew install awscli

# Verify - must report aws-cli/2.x
aws --version

# Smoke test against the real service
aws dynamodb list-tables --region us-east-1
```

Listing 4.1 — AWS CLI v2 installation. CLI v1 is end-of-life; do not build new tooling on it.

4.2 DynamoDB Local (Docker) — the developer's best friend

DynamoDB Local is an AWS-published Java emulator of the DynamoDB API. It costs nothing, runs offline, and is the correct target for unit tests and CI pipelines. It is **not** a production substitute: it does not emulate partitions, adaptive capacity, throttling behaviour, or latency.

```bash
# Fastest path: run the official image with on-disk persistence
docker run -d --name ddb-local -p 8000:8000 \
  -v "$PWD/ddb-data:/home/dynamodblocal/data" \
  amazon/dynamodb-local:latest \
  -jar DynamoDBLocal.jar -sharedDb -dbPath /home/dynamodblocal/data

# Health check (should return an empty TableNames array)
aws dynamodb list-tables --endpoint-url http://localhost:8000 --region us-east-1

# Create a table locally
aws dynamodb create-table \
  --endpoint-url http://localhost:8000 --region us-east-1 \
  --table-name Orders \
  --attribute-definitions AttributeName=PK,AttributeType=S AttributeName=SK,AttributeType=S \
  --key-schema AttributeName=PK,KeyType=HASH AttributeName=SK,KeyType=RANGE \
  --billing-mode PAY_PER_REQUEST
```

```
# Tear down
docker rm -f ddb-local
```

Listing 4.2 — DynamoDB Local via Docker. `-sharedDb` forces a single shared database file regardless of credentials/Region — without it, every distinct access key gets its own isolated database and your tests will 'lose' tables.

Table 4.1 — DynamoDB Local flags that matter

Flag	Effect	When to use
`-sharedDb`	One database file for all credentials/Regions	**Almost always.** Prevents phantom-missing tables
`-inMemory`	Nothing persists to disk; fastest	CI unit tests — clean state per run
`-dbPath <dir>`	Persist to a directory	Local dev where you want data to survive restarts
`-delayTransientStatuses`	Simulates CREATING/UPDATING delays	Testing code that polls table status
`-port <n>`	Listen on a different port	Running several isolated stacks

WARNING — Where DynamoDB Local lies to you

- It ignores provisioned capacity — you will **never** see a `ProvisionedThroughputExceededException`. Throttle handling must be tested against the real service.
- It has no partitions, so hot-key problems are invisible locally and catastrophic in production.
- It is single-threaded-ish and slow on large scans; performance numbers mean nothing.
- Streams, TTL and Global Tables behaviour are partially emulated at best. **Integration-test in a real sandbox account.**

4.3 Install the SDKs

BASH

```
# Python (boto3)
pip install boto3 botocore

# Node.js (AWS SDK v3 - modular; do NOT use v2, it is end-of-support)
npm install @aws-sdk/client-dynamodb @aws-sdk/lib-dynamodb

# .NET
dotnet add package AWSSDK.DynamoDBv2

# Java (Maven coordinates)
#   software.amazon.awssdk : dynamodb : 2.x
#   software.amazon.awssdk : dynamodb-enhanced : 2.x

# PowerShell (relevant for SQL Server DBAs automating alongside dbatools)
Install-Module -Name AWS.Tools.DynamoDBv2 -Scope CurrentUser -Force
```

Listing 4.3 — SDK installation across the common runtimes.

4.4 Create Your First Table — Three Ways

4.4.1 AWS CLI

```bash
aws dynamodb create-table \
  --table-name SDC-Orders \
  --attribute-definitions \
      AttributeName=PK,AttributeType=S \
      AttributeName=SK,AttributeType=S \
      AttributeName=GSI1PK,AttributeType=S \
      AttributeName=GSI1SK,AttributeType=S \
  --key-schema \
      AttributeName=PK,KeyType=HASH \
      AttributeName=SK,KeyType=RANGE \
  --global-secondary-indexes '[{
        "IndexName": "GSI1",
        "KeySchema": [
            {"AttributeName":"GSI1PK","KeyType":"HASH"},
            {"AttributeName":"GSI1SK","KeyType":"RANGE"}],
        "Projection": {"ProjectionType":"INCLUDE",
                       "NonKeyAttributes":["status","total","customerName"]}
      }]' \
  --billing-mode PAY_PER_REQUEST \
  --sse-specification 'Enabled=true,SSEType=KMS,KMSMasterKeyId=alias/sdc-ddb-cmk' \
  --stream-specification 'StreamEnabled=true,StreamViewType=NEW_AND_OLD_IMAGES' \
  --deletion-protection-enabled \
  --tags Key=Owner,Value=SQLDBAChamps Key=Env,Value=prod Key=CostCenter,Value=CC-4471

# Wait until ACTIVE (blocks until ready - use this in scripts)
aws dynamodb wait table-exists --table-name SDC-Orders

# Turn on Point-In-Time Recovery (a SEPARATE call - never forget it)
aws dynamodb update-continuous-backups \
  --table-name SDC-Orders \
  --point-in-time-recovery-specification PointInTimeRecoveryEnabled=true
```

Listing 4.4 — A production-shaped table in one command: composite key, a covering GSI, CMK encryption, Streams for CDC, deletion protection, tags, and PITR.

CRITICAL — `--attribute-definitions` is not a schema

You declare **only** the attributes used in a key schema (table keys and index keys). Declaring an attribute you do not use in a key throws `ValidationException`. Every other attribute is created implicitly at write time. This trips up every relational DBA on day one.

4.4.2 Terraform (the production path)

```hcl
resource "aws_dynamodb_table" "orders" {
  name         = "SDC-Orders"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "PK"
  range_key    = "SK"

  attribute { name = "PK"     type = "S" }
  attribute { name = "SK"     type = "S" }
  attribute { name = "GSI1PK" type = "S" }
  attribute { name = "GSI1SK" type = "S" }
```

```
  global_secondary_index {
    name               = "GSI1"
    hash_key           = "GSI1PK"
    range_key          = "GSI1SK"
    projection_type    = "INCLUDE"
    non_key_attributes = ["status", "total", "customerName"]
  }

  ttl {
    attribute_name = "expires_at"
    enabled        = true
  }

  stream_enabled   = true
  stream_view_type = "NEW_AND_OLD_IMAGES"

  point_in_time_recovery { enabled = true }

  server_side_encryption {
    enabled     = true
    kms_key_arn = aws_kms_key.ddb.arn
  }

  deletion_protection_enabled = true

  lifecycle {
    prevent_destroy = true            # belt and braces against terraform destroy
  }

  tags = {
    Owner      = "SQLDBAChamps"
    Env        = "prod"
    CostCenter = "CC-4471"
  }
}
```

Listing 4.5 — Terraform. Note both `deletion_protection_enabled` (service-side) and `prevent_destroy` (Terraform-side): they protect against different mistakes.

4.4.3 Python (boto3)

```python
import boto3
from botocore.config import Config

cfg = Config(
    region_name='us-east-1',
    retries={'max_attempts': 10, 'mode': 'adaptive'},   # client-side rate limiting
    max_pool_connections=50,
    connect_timeout=1, read_timeout=2
)
ddb = boto3.resource('dynamodb', config=cfg)

table = ddb.create_table(
    TableName='SDC-Orders',
    KeySchema=[
        {'AttributeName': 'PK', 'KeyType': 'HASH'},
        {'AttributeName': 'SK', 'KeyType': 'RANGE'},
    ],
    AttributeDefinitions=[
        {'AttributeName': 'PK', 'AttributeType': 'S'},
        {'AttributeName': 'SK', 'AttributeType': 'S'},
```

```
    ],
    BillingMode='PAY_PER_REQUEST',
    DeletionProtectionEnabled=True,
)
table.wait_until_exists()
print(table.table_status)   # ACTIVE
```

Listing 4.6 — boto3 with a production client config. `mode='adaptive'` is the retry mode you want for DynamoDB: it backs off client-side when throttled instead of amplifying the storm.

5. Configuration

Credentials, endpoints, client tuning, and every table-level setting you can turn on or off.

5.1 Credential Configuration

Table 5.1 — Credential resolution order (SDK default chain)

#	Source	Guidance
1	Explicit code parameters	**Never do this.** Hard-coded keys are the #1 breach vector
2	Env vars (`AWS_ACCESS_KEY_ID`, …)	Acceptable for CI where a short-lived role is exported
3	`~/.aws/credentials` profile	Fine for a workstation with SSO-sourced temporary creds
4	SSO / `sso-session`	**Preferred for humans.** `aws sso login`
5	Web identity token (IRSA)	**Preferred on EKS.** Pod-level IAM roles
6	ECS container credentials	**Preferred on ECS/Fargate.** Task role
7	EC2 IMDSv2 instance profile	**Preferred on EC2.** Enforce IMDSv2, hop-limit 1

```ini
# ~/.aws/config  - SSO + role chaining, zero long-lived keys
[sso-session sdc]
sso_start_url  = https://sqldbachamps.awsapps.com/start
sso_region     = us-east-1
sso_registration_scopes = sso:account:access

[profile ddb-prod]
sso_session    = sdc
sso_account_id = 111122223333
sso_role_name  = DynamoDBOperator
region         = us-east-1
output         = json

[profile ddb-local]
region     = us-east-1
endpoint_url = http://localhost:8000     # CLI v2.13+ honours this per-profile
```

Listing 5.1 — A clean credentials setup. `aws sso login --sso-session sdc` *then* `aws dynamodb list-tables --profile ddb-prod`.

5.2 Client-Side Configuration That Actually Moves the Needle

Table 5.2 — SDK settings for low-latency, throttle-resilient clients

Setting	Recommended	Why
Retry mode	`adaptive`	Adds client-side rate limiting on top of exponential backoff; prevents retry storms
Max attempts	5–10	DynamoDB throttles are transient; give the service room to scale
Connect timeout	1000 ms	Default 60s means a network blip stalls your thread pool for a minute

Setting	Recommended	Why
Read/socket timeout	1000–3000 ms	P99 for a point read is a few ms; anything slower is a fault, not slowness
Connection pool size	≥ max concurrency	Pool exhaustion presents as latency, and is misdiagnosed as DynamoDB being slow
Keep-alive (TCP)	Enabled	TLS handshake per request adds 10–40 ms; SDK v3 / boto3 enable it by default
Region endpoint	VPC Endpoint (Gateway)	Keeps traffic off the internet; no NAT Gateway data charges
Compression	gzip on large items	Application-side; a 400 KB JSON item often compresses to <100 KB, cutting RCU 4×

```javascript
// AWS SDK v3 (Node.js) - production client
import { DynamoDBClient } from '@aws-sdk/client-dynamodb';
import { DynamoDBDocumentClient } from '@aws-sdk/lib-dynamodb';
import { NodeHttpHandler } from '@smithy/node-http-handler';
import { Agent } from 'https';

const base = new DynamoDBClient({
  region: 'us-east-1',
  maxAttempts: 8,
  retryMode: 'adaptive',
  requestHandler: new NodeHttpHandler({
    connectionTimeout: 1000,
    requestTimeout: 2000,
    httpsAgent: new Agent({ keepAlive: true, maxSockets: 100 })
  })
});

// DocumentClient removes the {'S': 'value'} wire format from your code
export const ddb = DynamoDBDocumentClient.from(base, {
  marshallOptions: { removeUndefinedValues: true, convertClassInstanceToMap: true },
  unmarshallOptions: { wrapNumbers: false }
});
```

Listing 5.2 — Node.js SDK v3. `removeUndefinedValues: true` alone prevents a whole class of runtime errors.

5.3 Table-Level Configuration Reference

Table 5.3 — Every table setting, and the correct production default

Setting	Default	Recommended	Notes
Billing mode	Provisioned	On-Demand first, then evaluate	Switchable once per 24h
Auto Scaling target	n/a	70% utilisation	Only for Provisioned; min/max must bracket real traffic
PITR	**Disabled**	**Enabled everywhere**	35-day window; ~$0.20/GB-month; enable at create time
Deletion protection	**Disabled**	**Enabled on all prod tables**	Blocks `DeleteTable` even for an admin

Setting	Default	Recommended	Notes
Encryption at rest	AWS-owned key	**Customer-managed KMS key**	AWS-owned key cannot be audited or revoked
Streams	Disabled	Enabled (`NEW_AND_OLD_IMAGES`)	Required for Global Tables; enables CDC & audit later
TTL	Disabled	Enabled where data has a lifespan	Free deletes; huge storage saver
Contributor Insights	Disabled	Enable during tuning	Surfaces the top-N hot keys — chargeable
Table class	Standard	Standard-IA if >30 days cold	IA: ~60% cheaper storage, ~25% dearer throughput
Max throughput (On-Demand)	Unbounded	**Set a ceiling**	Your only defence against a runaway-loop bill
Tags	None	Owner, Env, CostCenter, DataClass	Drives cost allocation and IAM tag-based access

BASH

```
# --- Switch to Provisioned with Auto Scaling ---
aws dynamodb update-table --table-name SDC-Orders \
  --billing-mode PROVISIONED \
  --provisioned-throughput ReadCapacityUnits=100,WriteCapacityUnits=50

# Register the scalable target (writes)
aws application-autoscaling register-scalable-target \
  --service-namespace dynamodb \
  --resource-id "table/SDC-Orders" \
  --scalable-dimension "dynamodb:table:WriteCapacityUnits" \
  --min-capacity 50 --max-capacity 4000

# Attach the target-tracking policy at 70% utilisation
aws application-autoscaling put-scaling-policy \
  --service-namespace dynamodb \
  --resource-id "table/SDC-Orders" \
  --scalable-dimension "dynamodb:table:WriteCapacityUnits" \
  --policy-name SDC-Orders-WCU-70 \
  --policy-type TargetTrackingScaling \
  --target-tracking-scaling-policy-configuration '{
      "TargetValue": 70.0,
      "ScaleInCooldown": 300,
      "ScaleOutCooldown": 60,
      "PredefinedMetricSpecification": {
          "PredefinedMetricType": "DynamoDBWriteCapacityUtilization"}
  }'

# --- Guardrails ---
aws dynamodb update-table --table-name SDC-Orders --deletion-protection-enabled
aws dynamodb update-table --table-name SDC-Orders --table-class STANDARD_INFREQUENT_ACCESS
```

Listing 5.3 — Auto Scaling must be registered per dimension (table RCU, table WCU, and each GSI's RCU/WCU separately). Forgetting the GSI dimensions is a very common production gap.

WARNING — Auto Scaling is not instant

Application Auto Scaling reacts to CloudWatch metrics that arrive at 1-minute granularity, then needs a scale-out cooldown. Realistic reaction time is **2–7 minutes**. It protects you from a slow ramp; it will **not** protect you from a marketing email sent to 500,000 people at 09:00. For known spikes, pre-scale on a schedule (`--scheduled-action`) or run On-Demand.

5.4 VPC Endpoint Configuration

```bash
# Gateway endpoint - FREE, no data-processing charge, no NAT Gateway needed
aws ec2 create-vpc-endpoint \
  --vpc-id vpc-0abc123 \
  --service-name com.amazonaws.us-east-1.dynamodb \
  --vpc-endpoint-type Gateway \
  --route-table-ids rtb-0aaa111 rtb-0bbb222 \
  --policy-document file://endpoint-policy.json
```

Listing 5.4 — A Gateway endpoint keeps DynamoDB traffic on the AWS backbone and removes NAT Gateway data-processing costs, which for a chatty table can dwarf the DynamoDB bill itself.

```json
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "AllowOnlyOurTablesFromThisVPC",
    "Effect": "Allow",
    "Principal": "*",
    "Action": ["dynamodb:GetItem","dynamodb:PutItem","dynamodb:UpdateItem",
               "dynamodb:DeleteItem","dynamodb:Query","dynamodb:BatchGetItem",
               "dynamodb:BatchWriteItem"],
    "Resource": [
      "arn:aws:dynamodb:us-east-1:111122223333:table/SDC-*",
      "arn:aws:dynamodb:us-east-1:111122223333:table/SDC-*/index/*"
    ]
  }]
}
```

*Listing 5.5 — endpoint-policy.json. This is a **data-exfiltration control**: even a compromised instance with over-broad IAM cannot write your data to an attacker's table in another account through this endpoint.*

6. Prerequisites

What must exist — in the account, in the design, and in your head — before the first table is created.

6.1 Account & Platform Prerequisites

Table 6.1 — Environment checklist

Prerequisite	Detail	Verify with
AWS account + Region	DynamoDB is regional; the table name is unique per account+Region	`aws sts get-caller-identity`
IAM principal	A role (not a user) with least-privilege DynamoDB actions	`aws iam simulate-principal-policy`
AWS CLI v2 / SDK	CLI v1 and JS SDK v2 are end-of-support	`aws --version`
KMS CMK	Customer-managed key for at-rest encryption, with a key policy	`aws kms describe-key`
VPC endpoint	Gateway endpoint for private, cheaper access	`aws ec2 describe-vpc-endpoints`
CloudTrail	Management events on by default; enable data events selectively	`aws cloudtrail get-trail-status`
Service quotas	Tables/Region, GSIs/table, concurrent control-plane ops	`aws service-quotas list-service-quotas --service-code dynamodb`
Tagging standard	Owner / Env / CostCenter / DataClassification	AWS Config rule `required-tags`
IaC pipeline	Terraform or CDK — never click-ops a prod table	Repo + plan output

6.2 The Design Prerequisite — Access Pattern Enumeration

This is the prerequisite that people skip and then pay for over the following two years. **You cannot design a DynamoDB table without a complete, written list of access patterns.** In a relational database, a missed query is a new index. In DynamoDB, a missed access pattern can be a full table migration.

Table 6.2 — Access pattern worksheet (fill this in before writing any code)

#	Access pattern	Frequency	Index used	Key condition
1	Get customer profile by ID	Very high	Base table	`PK=CUST#<id>` AND `SK=PROFILE`
2	List a customer's orders, newest first	High	Base table	`PK=CUST#<id>` AND `begins_with(SK,'ORDER#')`, ScanIndexForward=false
3	Get an order + its line items	High	Base table	`PK=ORDER#<id>` (single Query returns the item collection)
4	Find all orders with status=PENDING	Medium	GSI1 (sparse)	`GSI1PK=PENDING`

#	Access pattern	Frequency	Index used	Key condition
5	Orders in a date range for a store	Medium	GSI2	`GSI2PK=STORE#<id>` AND `GSI2SK BETWEEN d1 AND d2`
6	Lookup order by external tracking no.	Low	GSI3	`GSI3PK=TRACK#<no>`
7	Monthly revenue by store	Batch	**Not DynamoDB**	S3 export → Athena

TIP — The single-table design decision

Single-table design — putting multiple entity types in one table with generic `PK`/`SK` attributes — minimises round trips and cost, and lets one `Query` return a parent and all its children in one request. It also makes the table harder to read, harder to hand to an analytics team, and harder for new engineers. **Rule of thumb: single-table when the access patterns are stable and latency/cost-critical; multi-table when the domain is still moving or the team is new to DynamoDB.** Both are defensible; being unable to explain the trade-off is not.

6.3 Service Quotas Worth Knowing

Table 6.3 — Limits you will actually hit

Quota	Value	Adjustable?
Item size	400 KB	**No — hard limit**
Partition key length	2048 bytes	No
Sort key length	1024 bytes	No
GSIs per table	20	Yes (soft)
LSIs per table	5	**No**
Item collection size (with LSI)	10 GB	**No**
`Query` / `Scan` response size	1 MB per page	**No** — you must paginate
`BatchGetItem`	100 items / 16 MB	No
`BatchWriteItem`	25 items / 16 MB	No
`TransactWriteItems`	100 items / 4 MB	No
Tables per Region	2,500	Yes (soft)
Concurrent table operations	500 CREATING/UPDATING/DELETING	Yes (soft)
Provisioned throughput per table	40,000 RCU / 40,000 WCU (per account default)	Yes (soft)
Expression length	4 KB	No
Nesting depth	32 levels	No

6.4 Skills Prerequisites for the Team

- **IAM policy authorship**, including condition keys such as `dynamodb:LeadingKeys` and `dynamodb:Attributes`.
- **Exponential backoff and idempotency** — the SDK retries by default; your `UpdateItem` must be safe to run twice.
- **CloudWatch metrics and alarms** — `ThrottledRequests` is not the same metric as `ThrottleEvents`; know which is which.
- **Cost reasoning** — an engineer who cannot estimate the RCU cost of a `Query` will design a bill, not a system.
- **Data modelling for NoSQL** — the mental shift from normalisation to denormalisation and pre-computed access paths.

7. Security Best Practices

Identity, encryption, network isolation, auditing, and the compliance controls that SOX / PCI-DSS auditors ask for by name.

MERMAID DIAGRAM — DEFENCE-IN-DEPTH LAYERS

```
flowchart TB
    CSE["Client-side encryption — AWS Database Encryption SDK<br/>PAN/PII encrypted BEFORE the
request leaves your process"]
    REQ["API request"]
    CSE -->|"service only ever sees ciphertext"| REQ

    subgraph INPATH["Evaluated on EVERY request — all must pass"]
        direction TB
        N["1. Network — VPC Gateway Endpoint<br/>endpoint policy · aws:SourceVpce"]
        TLS["2. Transport — TLS 1.2+<br/>aws:SecureTransport"]
        ID["3. Identity — SigV4 → IAM identity policy<br/>→ SCP → resource policy → LeadingKeys /
Attributes"]
        K["4. Encryption at rest — KMS CMK decrypt<br/>(the key policy must ALSO allow the
caller)"]
        N --> TLS --> ID --> K
    end

    REQ --> N
    K --> DATA[("Item read / written")]

    subgraph OUT["Out-of-band — always on, never in the request path"]
        direction TB
        DET["Detection — CloudTrail mgmt events (free)<br/>+ scoped data events · GuardDuty · AWS
Config"]
        AUD["Audit — Streams → S3 Object Lock (WORM)"]
        PRO["Data protection — PITR 35d · deletion protection<br/>Backup Vault Lock · cross-
account copy"]
    end

    DATA -.-> DET
    DATA -.-> AUD
    DATA -.-> PRO
```

Figure 7.1 — The four in-path controls are evaluated on every single call and any one of them can deny it — note that a KMS key policy denial looks exactly like an IAM problem and is the most-missed cause of AccessDenied. The out-of-band controls never block a request; they are what you rely on after one succeeds that should not have.

7.1 IAM — Least Privilege in Practice

JSON

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AppDataPlaneOnly",
      "Effect": "Allow",
      "Action": [
        "dynamodb:GetItem", "dynamodb:BatchGetItem", "dynamodb:Query",
        "dynamodb:PutItem", "dynamodb:UpdateItem", "dynamodb:DeleteItem",
        "dynamodb:BatchWriteItem", "dynamodb:ConditionCheckItem"
      ],
      "Resource": [
        "arn:aws:dynamodb:us-east-1:111122223333:table/SDC-Orders",
        "arn:aws:dynamodb:us-east-1:111122223333:table/SDC-Orders/index/*"
```

```
      ],
      "Condition": {
        "Bool": { "aws:SecureTransport": "true" },
        "StringEquals": { "aws:SourceVpce": "vpce-0abc123def456" }
      }
    },
    {
      "Sid": "NoScanNoDropNoExport",
      "Effect": "Deny",
      "Action": [
        "dynamodb:Scan",
        "dynamodb:DeleteTable",
        "dynamodb:DeleteBackup",
        "dynamodb:ExportTableToPointInTime",
        "dynamodb:UpdateContinuousBackups"
      ],
      "Resource": "*"
    }
  ]
}
```

Listing 7.1 — An application role. Note the explicit `Deny` on `Scan` (forces good design), on `DeleteTable`/`DeleteBackup` (blast radius), and on `ExportTableToPointInTime` (a legitimate, quiet exfiltration path that many teams forget to close).

7.1.1 Fine-Grained Access Control — row-level security in DynamoDB

`dynamodb:LeadingKeys` restricts a principal to items whose partition key matches an identity attribute. This is how a multi-tenant application lets a user's own credentials touch only that user's rows — enforced by IAM, not by application code.

JSON

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["dynamodb:GetItem","dynamodb:Query","dynamodb:PutItem","dynamodb:UpdateItem"],
    "Resource": "arn:aws:dynamodb:us-east-1:111122223333:table/SDC-UserData",
    "Condition": {
      "ForAllValues:StringEquals": {
        "dynamodb:LeadingKeys": ["${cognito-identity.amazonaws.com:sub}"],
        "dynamodb:Attributes": ["PK","SK","displayName","prefs","updatedAt"]
      },
      "StringEquals": { "dynamodb:Select": "SPECIFIC_ATTRIBUTES" }
    }
  }]
}
```

Listing 7.2 — Fine-grained access control. `LeadingKeys` = row-level security; `Attributes` + `Select=SPECIFIC_ATTRIBUTES` = column-level security. Together they are the closest DynamoDB gets to `GRANT SELECT(col) ... WHERE tenant_id = USER()`.

Table 7.1 — DynamoDB IAM condition keys

Condition key	Restricts	Compliance use
`dynamodb:LeadingKeys`	Which partition keys a principal may touch	Multi-tenant isolation (SOC 2, PCI segmentation)
`dynamodb:Attributes`	Which attributes may be read/written	Column-level PII control
`dynamodb:Select`	Forces `SPECIFIC_ATTRIBUTES`	Prevents `SELECT *`-style over-fetch
`dynamodb:ReturnValues`	Blocks returning old images	Stops leakage of pre-update PII

Condition key	Restricts	Compliance use
`dynamodb:EnclosingOperation`	Whether the call is inside a transaction	Forces atomicity for financial writes
`aws:SourceVpce`	Only from a named VPC endpoint	Network isolation evidence
`aws:SecureTransport`	TLS only	PCI-DSS Req. 4
`aws:PrincipalTag` / `aws:ResourceTag`	ABAC on tags	Scalable env/team separation

7.2 Encryption

Table 7.2 — Encryption-at-rest key options

Key type	Behaviour	Verdict
AWS owned key (default)	AWS-managed, invisible, free, no CloudTrail on key use	**Not acceptable** for regulated data — you cannot prove or revoke anything
AWS managed key (`aws/dynamodb`)	In your account, KMS-audited, free, cannot set the policy	Acceptable minimum; no key policy control
Customer managed key (CMK)	Your key policy, your rotation, your CloudTrail, ~$1/month + API calls	**The standard for SOX / PCI-DSS / HIPAA.** Enables cryptographic revocation

CRITICAL — The CMK kill-switch is real — and cuts both ways

Disabling or scheduling deletion of the CMK renders the table **completely inaccessible** within minutes. That is a genuine incident-response control (revoke access to a compromised dataset instantly) and a genuine self-inflicted-outage risk. Protect the CMK with an SCP that denies `kms:ScheduleKeyDeletion` and `kms:DisableKey` to everyone except a break-glass role, and alarm on both events.

7.2.1 Client-Side Encryption for PAN / PII

Server-side encryption protects against physical media theft. It does **not** protect against an over-permissioned application role, because the service decrypts transparently for any authorised caller. For cardholder data under PCI-DSS, or for the strictest SOX-scoped attributes, encrypt client-side with the **AWS Database Encryption SDK for DynamoDB** so the ciphertext is what lands in the table.

Table 7.3 — Attribute actions in the AWS Database Encryption SDK

Action	Meaning
`ENCRYPT_AND_SIGN`	Encrypted + integrity-protected. Use for PAN, SSN, DOB, salary.
`SIGN_ONLY`	Plaintext but tamper-evident. Use for keys and attributes you must Query on.
`DO_NOTHING`	Ignored entirely. Use for non-sensitive, non-authenticated data.

WARNING — The client-side encryption trade-off

You cannot `Query`, filter, or index an encrypted attribute — the service only sees ciphertext. Key attributes must therefore be `SIGN_ONLY`. If you need to look something up by an encrypted value, store a deterministic HMAC of it in a separate `SIGN_ONLY` attribute and index that. Searchable encryption (beacons) in the Database Encryption SDK does exactly this — and leaks some frequency information, which must be documented in your threat model.

7.3 Auditing & Compliance

Table 7.4 — Mapping DynamoDB controls to audit requirements

Requirement	Control	Evidence artefact
SOX — change control	IaC + deletion protection + CloudTrail management events	Terraform plan/apply logs + CloudTrail `UpdateTable` records
SOX — segregation of duties	Separate IAM roles for app / DBA / auditor; deny `DeleteTable` to all but break-glass	IAM policy export + `simulate-principal-policy` output
PCI-DSS Req. 3 — protect stored PAN	Client-side encryption + CMK + no PAN in keys/GSIs	Encryption SDK config + key policy
PCI-DSS Req. 4 — encrypt in transit	`aws:SecureTransport` deny-if-false	IAM/SCP policy document
PCI-DSS Req. 10 — audit trails	CloudTrail **data events** for the CDE table + Streams→S3 Object Lock	CloudTrail event selectors + S3 Object Lock config
Req. 10.5 — protect logs	S3 Object Lock (WORM) + separate log archive account	Bucket policy + Object Lock retention
Ransomware / insider	AWS Backup Vault Lock in compliance mode, cross-account copy	Vault Lock configuration
Data residency	Region pinning + SCP `aws:RequestedRegion` deny	SCP document

```bash
# CloudTrail DATA events for one table only - full-account data events are ruinously expensive
aws cloudtrail put-event-selectors \
  --trail-name sdc-audit-trail \
  --advanced-event-selectors '[{
     "Name": "DynamoDB data events for the CDE table",
     "FieldSelectors": [
        {"Field": "eventCategory", "Equals": ["Data"]},
        {"Field": "resources.type", "Equals": ["AWS::DynamoDB::Table"]},
        {"Field": "resources.ARN", "StartsWith":
             ["arn:aws:dynamodb:us-east-1:111122223333:table/SDC-Payments"]}
     ]}]'

# Who deleted / altered a table in the last 7 days?
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=EventName,AttributeValue=DeleteTable \
  --start-time "$(date -u -d '7 days ago' +%Y-%m-%dT%H:%M:%SZ)" \
  --query 'Events[].{Time:EventTime,User:Username,Res:Resources[0].ResourceName}' \
  --output table
```

Listing 7.3 — Scoped data-event logging. Turning on data events account-wide is one of the classic ways to accidentally quadruple a CloudTrail bill.

7.4 Security Checklist

Table 7.5 — Pre-production security gate

✓	Control	Verify
☐	No IAM users with long-lived keys touch DynamoDB — roles only	IAM Credential Report
☐	No `dynamodb:*` and no `Resource: "*"` in any application policy	IAM Access Analyzer
☐	`Scan` and `DeleteTable` explicitly denied to application roles	Policy review
☐	Encryption at rest uses a customer-managed KMS key	`describe-table` → SSEDescription
☐	`aws:SecureTransport` = true enforced	SCP / policy review
☐	VPC Gateway Endpoint in place with a restrictive endpoint policy	`describe-vpc-endpoints`
☐	Deletion protection ON for every production table	`describe-table`
☐	PITR ON for every production table	`describe-continuous-backups`
☐	AWS Backup plan + Vault Lock + cross-account copy	Backup plan review
☐	CloudTrail data events on tables holding regulated data	`get-event-selectors`
☐	CMK deletion/disable denied by SCP, alarmed in CloudWatch	SCP + alarm
☐	PII/PAN encrypted client-side; never used as a key or GSI key	Data model review
☐	Every table tagged: Owner, Env, CostCenter, DataClassification	AWS Config `required-tags`
☐	AWS Config rules: `dynamodb-pitr-enabled`, `dynamodb-table-encrypted-kms`	Config dashboard

INTERVIEW ANGLE — Interview angle

"How do you implement row-level security in DynamoDB?" — The answer is `dynamodb:LeadingKeys` with a Cognito identity variable, combined with `dynamodb:Attributes` for column-level control. Add the nuance: *"That pattern only works when the tenant identifier is the leading component of the partition key, which is a design decision you must make on day one. It's another example of DynamoDB pushing decisions to the front of the project."*

8. Performance Tuning

There is no query optimiser, no plan cache and no index rebuild. Performance in DynamoDB is decided by key design, and everything else is a rounding error.

> **NOTE — The tuning hierarchy**
> **1. Key design** (90% of the outcome) → **2. Query vs. Scan** → **3. Item size and projections** → **4. Capacity mode & Auto Scaling** → **5. Client config (batching, parallelism, keep-alive)** → **6. DAX**. Teams routinely start at step 6 and wonder why nothing improves. If your partition key is wrong, DAX just caches a hot spot.

8.1 Capacity Unit Arithmetic — learn this cold

Table 8.1 — RCU / WCU rules

Operation	Rounding	Cost formula
Strongly consistent read	Up to 4 KB blocks	`RCU = ceil(itemKB / 4)`
Eventually consistent read	Up to 4 KB blocks	`RCU = ceil(itemKB / 4) / 2`
Transactional read	Up to 4 KB blocks	`RCU = 2 × ceil(itemKB / 4)`
Standard write	Up to 1 KB blocks	`WCU = ceil(itemKB / 1)`
Transactional write	Up to 1 KB blocks	`WCU = 2 × ceil(itemKB / 1)`
`Query`	Sum of all **returned** items, rounded once	Efficient — 100 × 200-byte items ≈ 20 KB → 5 RCU strong
`Scan`	Sum of all **examined** items	`FilterExpression` does **not** reduce cost
GSI write	Full WCU per index touched	A 1 KB item + 3 GSIs (ALL) ≈ 4 WCU per write

```text
WORKED EXAMPLE - sizing a 'customer orders' page
------------------------------------------------
Requirement : 500 page-loads/sec, each returning ~25 orders of ~1.5 KB

Per page-load payload  = 25 x 1.5 KB          = 37.5 KB
Query rounds ONCE on the total (not per item) = ceil(37.5 / 4) = 10 RCU strong
Eventually consistent                          = 10 / 2        =  5 RCU
Sustained requirement                          = 500 x 5       = 2,500 RCU

  --> 2,500 RCU is under the 3,000 RCU/partition ceiling, BUT only if the
      load spreads across many customers. If one customer is 40% of traffic:
      1,000 RCU on one partition key -> fine. If it were 4,000 -> hot partition.

COUNTER-EXAMPLE - the same data fetched with 25 x GetItem instead of 1 x Query
  Each GetItem rounds INDIVIDUALLY: ceil(1.5/4) = 1 RCU strong -> 0.5 eventual
  25 x 0.5 = 12.5 RCU per page-load -> 500 x 12.5 = 6,250 RCU

  --> 2.5x the cost and 25x the network round trips, for identical data.
      THIS is why Query beats GetItem loops, and why item collections matter.
```

Listing 8.1 — Why one `Query` beats twenty-five `GetItem` calls: rounding happens once per request, not once per item.

8.2 Hot Partitions — the #1 production performance problem

MERMAID DIAGRAM — HOT PARTITION VS. EVEN DISTRIBUTION

```
flowchart TB
    subgraph Bad["❌ Hot partition — PK = order_date"]
        BW["10,000 writes/sec"] --> BP1["Partition A<br/>PK='2026-07-15'<br/>10,000 WCU needed<br/>1,000 WCU available"]
        BP1 --> BT["ProvisionedThroughputExceededException<br/>even at 40,000 table WCU"]
        BP2["Partition B — idle"]
        BP3["Partition C — idle"]
    end
    subgraph Good["☑ Write sharding — PK = order_date#shard"]
        GW["10,000 writes/sec"] --> S0["'2026-07-15#00' → 1,000 WCU"]
        GW --> S1["'2026-07-15#01' → 1,000 WCU"]
        GW --> S2["'2026-07-15#02' → 1,000 WCU"]
        GW --> SN["... #09 → 1,000 WCU"]
        S0 --> GT["10 partitions × 1,000 WCU = 10,000 WCU ✓"]
        S1 --> GT
        S2 --> GT
        SN --> GT
    end
```

Figure 8.1 — A table can have 40,000 WCU provisioned and still throttle if one partition key absorbs the traffic. Capacity is per-partition, not per-table.

Table 8.2 — Partition key selection

Candidate key	Verdict	Reason
`user_id`, `order_id`, `device_id` (UUID)	**Excellent**	High cardinality, uniform access
`tenant_id` in a 5-tenant SaaS	**Poor**	Cardinality 5; one big tenant = permanent hot partition
`order_date`	**Terrible**	All of today's traffic lands on one key
`status` (`PENDING`/`SHIPPED`)	**Terrible**	Cardinality 2. Classic 'looks fine in dev' disaster
`country_code`	**Poor**	Zipfian: `US` gets 60% of the traffic
`tenant_id#user_id`	**Good**	Preserves tenancy while lifting cardinality
`order_date#shard(0..N)`	**Good**	Write sharding — the standard fix for time-series

PYTHON

```python
# WRITE SHARDING - deterministic (recommended over random)
# Deterministic shards mean you can go BACK to a specific item without scanning.
import hashlib

SHARDS = 10

def shard_for(order_id: str) -> int:
    h = hashlib.md5(order_id.encode()).hexdigest()
    return int(h, 16) % SHARDS

def put_order(table, order_id, order_date, payload):
    table.put_item(Item={
        'PK': f'{order_date}#{shard_for(order_id):02d}',   # e.g. 2026-07-15#07
        'SK': f'ORDER#{order_id}',
        **payload
    })
```

```
# READ FAN-OUT - query every shard in parallel, then merge
from concurrent.futures import ThreadPoolExecutor
from boto3.dynamodb.conditions import Key

def orders_for_date(table, order_date):
    def q(shard):
        return table.query(
            KeyConditionExpression=Key('PK').eq(f'{order_date}#{shard:02d}')
        )['Items']
    with ThreadPoolExecutor(max_workers=SHARDS) as ex:
        pages = ex.map(q, range(SHARDS))
    return [i for page in pages for i in page]

# TRADE-OFF: writes scale linearly; reads now cost N queries instead of 1.
# Only shard when the write hot spot is real - measured, not imagined.
```

Listing 8.2 — Write sharding. Use `SHARDS = ceil(required_WCU / 1000)` and round up generously; changing the shard count later means rewriting historical data.

8.2.1 Adaptive Capacity — what it does and does not do

- **Instant adaptive capacity** lets one partition borrow unused capacity from the table's other partitions, automatically and for free. It handles moderate, transient imbalance well.
- **Split for heat** goes further: it detects a persistently hot key range and splits the partition to isolate it.
- **It cannot help a single partition key that needs more than 1,000 WCU / 3,000 RCU.** A key is indivisible. If one key is the hot spot, only write sharding, DAX (reads) or a redesign will fix it.
- It reacts in minutes, not milliseconds — a sudden spike still throttles briefly.

8.3 Query vs. Scan

Table 8.3 — Why `Scan` is almost always a bug

Aspect	`Query`	`Scan`
Cost basis	Only the items returned	**Every item in the table**
Requires	Equality on the partition key	Nothing
Latency	O(items returned)	O(table size)
Effect of `FilterExpression`	Reduces payload; you still pay for what was read	**Zero cost saving** — filter runs after the read
Capacity behaviour	Bounded	Consumes the whole table's capacity; starves live traffic
Legitimate uses	Everything in an OLTP path	One-off migrations, full exports (prefer S3 Export), tiny config tables

CRITICAL — The 3 a.m. Scan incident

A nightly reporting job runs `Scan` on a 400 GB table. It consumes 100% of the table's capacity for 40 minutes. Customer-facing `GetItem` calls throttle. The on-call engineer sees `ThrottledRequests` and raises provisioned capacity — which splits the table into more partitions permanently, diluting throughput. **Fix**: (1) delete the Scan, (2) use `ExportTableToPointInTime` to S3 + Athena — it consumes **zero** table capacity and

does not touch the live table at all, (3) if a Scan is truly unavoidable, rate-limit it and run it against a segment with `Limit` and a token bucket.

```python
# If you MUST Scan: parallel + rate-limited + returning only what you need
import boto3, time, itertools
from concurrent.futures import ThreadPoolExecutor

TOTAL_SEGMENTS = 8          # match to your consumer parallelism, not the partition count
MAX_RCU_PER_SEC = 200       # a hard ceiling so live traffic survives

table = boto3.resource('dynamodb').Table('SDC-Orders')

def scan_segment(segment: int):
    kwargs = {
        'Segment': segment,
        'TotalSegments': TOTAL_SEGMENTS,
        'ProjectionExpression': 'PK, SK, #s, #t',      # never fetch what you won't use
        'ExpressionAttributeNames': {'#s': 'status', '#t': 'total'},
        'ReturnConsumedCapacity': 'TOTAL',
        'Limit': 100,
    }
    budget, window = MAX_RCU_PER_SEC / TOTAL_SEGMENTS, time.time()
    spent = 0
    while True:
        resp = table.scan(**kwargs)
        yield from resp['Items']
        spent += resp['ConsumedCapacity']['CapacityUnits']
        if time.time() - window >= 1:
            spent, window = 0, time.time()
        elif spent >= budget:
            time.sleep(max(0, 1 - (time.time() - window)))
            spent, window = 0, time.time()
        if 'LastEvaluatedKey' not in resp:
            break
        kwargs['ExclusiveStartKey'] = resp['LastEvaluatedKey']

with ThreadPoolExecutor(max_workers=TOTAL_SEGMENTS) as ex:
    for item in itertools.chain.from_iterable(
            ex.map(lambda s: list(scan_segment(s)), range(TOTAL_SEGMENTS))):
        pass  # process(item)
```

Listing 8.3 — Parallel Scan with a token-bucket rate limit and a projection. This is the least bad Scan; the best Scan is the one you deleted.

8.4 Item Size Optimisation

Table 8.4 — Cheap wins on item size

Technique	Saving	Cost / caveat
Short attribute names (`cid` not `customerIdentifier`)	Attribute names count toward the 400 KB limit **and** toward WCU on every write	Readability; mitigate with a documented dictionary
gzip large text/JSON blobs into a `B` attribute	Often 60–80% size reduction → proportional RCU/WCU reduction	Cannot filter/index the compressed attribute
Move blobs >100 KB to S3, keep the key	Item drops from 100 RCU to <1 RCU per read	Two round trips; eventual consistency between the two stores

Technique	Saving	Cost / caveat
Vertical partitioning (hot vs. cold attributes as separate items)	Hot reads stop paying for cold data	Writes may need a transaction
`ProjectionExpression` on every read	Cuts network and CPU (not RCU — RCU is based on the stored item)	None. Always do it.
`INCLUDE` rather than `ALL` projections on GSIs	Cuts GSI storage and write WCU	A missing attribute forces a base-table fetch

TIP — The `ProjectionExpression` nuance interviewers love

`ProjectionExpression` reduces **network payload and latency**, but not RCU — capacity is charged on the size of the item as stored, before projection. The only way to reduce read *cost* is to reduce the stored item size, or read from a GSI with a narrower projection. Knowing this distinction is a reliable senior-level signal.

8.5 Batching, Parallelism, and Transactions

Table 8.5 — Choosing the right write API

API	Max	Atomic?	Use when
`PutItem` / `UpdateItem`	1 item	Per item	Default. Add a `ConditionExpression` for optimistic concurrency
`BatchWriteItem`	25 items / 16 MB	**No** — partial failure returns `UnprocessedItems`	Bulk ingest. **You must retry `UnprocessedItems` with backoff — the SDK does not**
`TransactWriteItems`	100 items / 4 MB	**Yes** — all-or-nothing	Financial writes, cross-entity invariants. Costs 2× WCU
`ExecuteStatement` (PartiQL)	1 statement	Per item	Ad-hoc console work. **Avoid in application code** — hides Scans behind SQL syntax

```python
# The UnprocessedItems bug that ships to production more often than any other
import boto3, random, time

client = boto3.client('dynamodb')

def batch_write_correctly(table_name, requests):
    """requests: list of {'PutRequest': {...}} - handles the 25-item cap AND retries."""
    for i in range(0, len(requests), 25):
        batch = requests[i:i + 25]
        unprocessed = {table_name: batch}
        attempt = 0
        while unprocessed:
            resp = client.batch_write_item(RequestItems=unprocessed)
            unprocessed = resp.get('UnprocessedItems', {})
            if unprocessed:
                attempt += 1
```

```
                if attempt > 10:
                    raise RuntimeError(f'Gave up with {len(unprocessed[table_name])} items
unwritten')
                # Full jitter backoff - do NOT hammer a throttling table
                time.sleep(random.uniform(0, min(30, 0.05 * (2 ** attempt))))

# A 200 OK from batch_write_item does NOT mean the data was written.
# Ignoring UnprocessedItems = silent, permanent data loss.
```

Listing 8.4 — `BatchWriteItem` returns HTTP 200 even when it wrote nothing. Every unhandled `UnprocessedItems` is a silent data-loss bug.

```python
# Optimistic concurrency - the DynamoDB answer to 'UPDATE ... WHERE version = @v'
from botocore.exceptions import ClientError

def debit_balance(table, account_id, amount, expected_version):
    try:
        return table.update_item(
            Key={'PK': f'ACCT#{account_id}', 'SK': 'BALANCE'},
            UpdateExpression='SET #b = #b - :amt, #v = #v + :one',
            ConditionExpression='#v = :ver AND #b >= :amt',   # <-- the whole point
            ExpressionAttributeNames={'#b': 'balance', '#v': 'version'},
            ExpressionAttributeValues={':amt': amount, ':ver': expected_version, ':one': 1},
            ReturnValues='ALL_NEW'
        )['Attributes']
    except ClientError as e:
        if e.response['Error']['Code'] == 'ConditionalCheckFailedException':
            raise ConcurrentModification('Stale version or insufficient funds')
        raise

# A failed ConditionExpression still costs 1 WCU. That is the correct price for
# correctness - but it means a hot contended item burns capacity on failures.
```

Listing 8.5 — Conditional writes give you atomicity and optimistic locking without a transaction, at 1× WCU instead of 2×.

8.6 Performance Tuning Checklist

Table 8.6 — Diagnostic decision table

Symptom	Likely cause	Action
Throttling well below provisioned capacity	Hot partition key	Contributor Insights → identify key → shard or redesign
Throttling only on writes, base table looks fine	**GSI throttling backpressuring the base table**	Raise GSI capacity or switch the GSI to On-Demand
Latency spikes every N minutes	Auto Scaling lagging a periodic spike	Raise the minimum, add a scheduled action, or use On-Demand
High RCU, low returned rows	Scan with a FilterExpression	Add a GSI to serve the pattern; delete the Scan
P99 latency 10× P50	Client-side: connection-pool exhaustion or DNS	Raise `max_pool_connections`; enable keep-alive
Cost per read is very high	Large items with 4 KB read rounding	Compress; move blobs to S3; vertically partition
Write cost 4× the expected	Multiple GSIs with `ALL` projection	Switch to `INCLUDE`; delete unused GSIs

Symptom	Likely cause	Action
Throughput dropped after scaling down	Throughput dilution from a past provisioning peak	Wait for adaptive capacity, or rebuild the table

9. Monitoring

The metrics that matter, the alarms that page you, and the dashboard you should build on day one.

9.1 Essential CloudWatch Metrics

Table 9.1 — DynamoDB CloudWatch metrics (namespace `AWS/DynamoDB`)

Metric	Statistic	Meaning	Alarm threshold
`ThrottledRequests`	Sum	Requests rejected — the top-line pain signal	**> 0 for 2 min → page**
`ReadThrottleEvents`	Sum	Per-request read throttles (incl. per index)	> 0 sustained
`WriteThrottleEvents`	Sum	Per-request write throttles	> 0 sustained
`ConsumedReadCapacityUnits`	Sum	Actual RCU burn	> 80% of provisioned
`ConsumedWriteCapacityUnits`	Sum	Actual WCU burn	> 80% of provisioned
`ProvisionedReadCapacityUnits`	Average	What you are paying for	Trend for cost review
`SuccessfulRequestLatency`	Average, p99	Server-side latency by operation	p99 > 20 ms (point read)
`SystemErrors`	Sum	HTTP 5xx from the service	> 5 in 5 min
`UserErrors`	Sum	HTTP 4xx — usually **your bug**	Spike vs. baseline
`ConditionalCheckFailedRequests`	Sum	Optimistic-lock collisions	Spike = contention hot spot
`TransactionConflict`	Sum	Transaction contention	> 0 sustained
`ReturnedItemCount`	Sum	Items returned by Query/Scan	Compare against consumed RCU
`AccountProvisionedReadCapacityUtilization`	Average	Account-level headroom	> 80%
`OnlineIndexPercentageProgress`	Average	GSI backfill progress	Watch during index adds
`AgeOfOldestUnreplicatedRecord`	Maximum	**Global Tables replication lag**	**> 60,000 ms → page**
`ReplicationLatency`	Average	Cross-Region lag	> 3 s
`PendingReplicationCount`	Maximum	Backlog of unreplicated writes	Rising trend
`TimeToLiveDeletedItemCount`	Sum	TTL activity	Drop to 0 = TTL config bug

Metric	Statistic	Meaning	Alarm threshold
`GetRecords.IteratorAgeMilliseconds`	Maximum	**Stream consumer lag** (via Lambda)	**> 4 h → page (24 h = data loss)**

WARNING — `ThrottledRequests` vs. `ReadThrottleEvents` — the distinction that catches people

A `BatchGetItem` for 100 items where 3 items throttle counts as **1** `ThrottledRequests` but **3** `ReadThrottleEvents`. Alarm on `ThrottleEvents` for capacity tuning (it reflects true magnitude) and on `ThrottledRequests` for user impact (it reflects how many client calls degraded). Alarming on only one of them gives you an incomplete picture.

9.2 Alarms Worth Paging For

BASH

```
SNS=arn:aws:sns:us-east-1:111122223333:sdc-dba-oncall
TBL=SDC-Orders

# 1. ANY throttling for 2 consecutive minutes -> customer impact
aws cloudwatch put-metric-alarm \
  --alarm-name "DDB-${TBL}-Throttled" \
  --namespace AWS/DynamoDB --metric-name ThrottledRequests \
  --dimensions Name=TableName,Value=${TBL} \
  --statistic Sum --period 60 --evaluation-periods 2 \
  --threshold 0 --comparison-operator GreaterThanThreshold \
  --treat-missing-data notBreaching \
  --alarm-actions ${SNS}

# 2. Service-side 5xx
aws cloudwatch put-metric-alarm \
  --alarm-name "DDB-${TBL}-SystemErrors" \
  --namespace AWS/DynamoDB --metric-name SystemErrors \
  --dimensions Name=TableName,Value=${TBL} \
  --statistic Sum --period 300 --evaluation-periods 1 \
  --threshold 5 --comparison-operator GreaterThanThreshold \
  --alarm-actions ${SNS}

# 3. Global Tables replication lag > 60s
aws cloudwatch put-metric-alarm \
  --alarm-name "DDB-${TBL}-ReplicationLag" \
  --namespace AWS/DynamoDB --metric-name AgeOfOldestUnreplicatedRecord \
  --dimensions Name=TableName,Value=${TBL} Name=ReceivingRegion,Value=eu-west-1 \
  --statistic Maximum --period 60 --evaluation-periods 3 \
  --threshold 60000 --comparison-operator GreaterThanThreshold \
  --alarm-actions ${SNS}

# 4. Stream consumer falling behind (4h of the 24h budget consumed)
aws cloudwatch put-metric-alarm \
  --alarm-name "DDB-${TBL}-StreamIteratorAge" \
  --namespace AWS/Lambda --metric-name IteratorAge \
  --dimensions Name=FunctionName,Value=sdc-orders-stream-consumer \
  --statistic Maximum --period 300 --evaluation-periods 2 \
  --threshold 14400000 --comparison-operator GreaterThanThreshold \
  --alarm-actions ${SNS}
```

Listing 9.1 — The four alarms every production DynamoDB table needs before it takes traffic.

9.3 Contributor Insights — finding the hot key

CloudWatch Contributor Insights for DynamoDB is the only first-party tool that tells you **which key** is hot. Without it, diagnosing a hot partition means guessing.

BASH

```
# Enable on the table and its indexes
aws dynamodb update-contributor-insights \
  --table-name SDC-Orders \
  --contributor-insights-action ENABLE

aws dynamodb update-contributor-insights \
  --table-name SDC-Orders --index-name GSI1 \
  --contributor-insights-action ENABLE

# The four rules it creates:
#   MostAccessedKeys       - top partition keys by traffic          <- hot partition
#   MostAccessedItems      - top full keys (PK+SK)                  <- hot item
#   MostThrottledKeys      - top partition keys by throttle count   <- the smoking gun
#   MostThrottledItems     - top items by throttle count

aws cloudwatch get-insight-rule-report \
  --rule-name "DynamoDBContributorInsights-PKC-SDC-Orders-..." \
  --start-time "$(date -u -d '1 hour ago' +%Y-%m-%dT%H:%M:%SZ)" \
  --end-time "$(date -u +%Y-%m-%dT%H:%M:%SZ)" \
  --period 60 --max-contributor-count 10
```

Listing 9.2 — Contributor Insights. Turn it on when tuning; it is chargeable, so many teams enable it on demand rather than permanently.

9.4 The Dashboard

MERMAID DIAGRAM — MONITORING ARCHITECTURE

```
flowchart LR
    DDB[("DynamoDB Table")] -->|"metrics (1 min)"| CW["CloudWatch Metrics"]
    DDB -->|"top-N keys"| CI["Contributor Insights"]
    DDB -->|"item changes"| STR["DynamoDB Streams"]
    DDB -->|"API calls"| CT["CloudTrail"]

    CW --> ALM["CloudWatch Alarms"]
    CW --> DASH["CloudWatch Dashboard"]
    CI --> DASH
    ALM --> SNS["SNS Topic"]
    SNS --> PD["PagerDuty / on-call"]
    SNS --> SLK["Slack #sdc-alerts"]

    STR --> LAM["Lambda consumer"]
    LAM --> S3["S3 audit archive (Object Lock)"]
    LAM --> OS["OpenSearch (search)"]

    CT --> SEC["Security Lake / SIEM"]
    CT --> ATH["Athena (forensics)"]

    DDB -->|"full export, 0 RCU"| EXP["S3 Export (PITR)"]
    EXP --> ATH
```

Figure 9.1 — Metrics for alerting, Contributor Insights for hot keys, Streams for audit/search, CloudTrail for forensics, S3 Export for analytics without touching table capacity.

Table 9.2 — Dashboard widgets, in priority order

#	Widget	What you are looking for
1	ThrottledRequests (Sum) — table + every GSI	The single most important line. GSIs must be plotted separately
2	Consumed vs. Provisioned RCU/WCU	Headroom and Auto Scaling behaviour
3	SuccessfulRequestLatency p50/p99 by operation	Divergence = client-side or item-size problem
4	SystemErrors + UserErrors	5xx = AWS; 4xx = your code
5	Contributor Insights: MostThrottledKeys	Names the guilty partition key
6	Stream IteratorAge	Approaching the 24h cliff
7	AgeOfOldestUnreplicatedRecord per Region	Global Tables health
8	ConditionalCheckFailed + TransactionConflict	Contention on a hot item
9	Estimated cost by table (Cost Explorer widget)	The bill you will be asked about

10. Logging

DynamoDB has no error log, no SQL trace, and no Extended Events. Here is where the truth actually lives.

> **NOTE — Reset your expectations**
> As a SQL Server DBA you reach for the ERRORLOG, the default trace, XEvents, or Query Store. DynamoDB has **none** of these. Observability is assembled from four external sources: **CloudTrail** (who called what), **CloudWatch Metrics** (how much and how fast), **DynamoDB Streams** (what the data became), and **your application logs** (why). If you did not turn them on before the incident, the evidence does not exist.

Table 10.1 — Where each question gets answered

Question	Source	Enabled by default?	Retention
Who deleted my table?	CloudTrail management events	**Yes** (90-day Event history)	90 days free; ∞ in an S3 trail
Who read customer 1234's PII?	CloudTrail **data events**	**No** — must be enabled, chargeable	As configured in S3
What did the item look like before the update?	Streams `OLD_IMAGE`	No	**24 hours only**
Why is P99 latency 200 ms?	CloudWatch metrics + client logs	Metrics: yes	15 months
Which key is hot?	Contributor Insights	No	Rolling 24h window
What did my app actually send?	SDK/app logs + X-Ray	No	Yours
Full history of every item change	Streams → S3 (Object Lock)	No — you build it	Whatever you set

10.1 CloudTrail — Management vs. Data Events

Table 10.2 — The two event categories

Aspect	Management events	Data events
Captures	CreateTable, UpdateTable, DeleteTable, CreateBackup, RestoreTable…	GetItem, PutItem, Query, Scan, UpdateItem, DeleteItem…
Default	**On, free**, 90-day Event history	**Off**, chargeable per event
Volume	A handful per day	**Millions per hour** on a busy table
Cost	Effectively free	Can exceed the DynamoDB bill itself
Use	Change control, SOX evidence	PCI-DSS Req.10, forensic investigation, insider-threat detection
Recommendation	Always on, to a locked S3 bucket in a log-archive account	**Scoped to tables holding regulated data only**

BASH

```
# Forensics with Athena over a CloudTrail S3 trail
# (After creating the CloudTrail Athena table via the console's 'Create table for CloudTrail
logs')

# Q1: Every control-plane change to DynamoDB in the last 30 days - the SOX report
SELECT eventtime, useridentity.arn AS who, eventname,
       json_extract_scalar(requestparameters,'$.tableName') AS table_name,
       sourceipaddress, errorcode
FROM   cloudtrail_logs
WHERE  eventsource = 'dynamodb.amazonaws.com'
  AND  eventname IN ('CreateTable','UpdateTable','DeleteTable','DeleteBackup',
                     'UpdateContinuousBackups','UpdateTimeToLive','RestoreTableFromBackup')
  AND  from_iso8601_timestamp(eventtime) > current_timestamp - interval '30' day
ORDER  BY eventtime DESC;

# Q2: Anyone Scanning the payments table (should be nobody)
SELECT useridentity.arn AS who, count(*) AS scans,
       min(eventtime) AS first_seen, max(eventtime) AS last_seen
FROM   cloudtrail_logs
WHERE  eventsource = 'dynamodb.amazonaws.com'
  AND  eventname = 'Scan'
  AND  json_extract_scalar(requestparameters,'$.tableName') = 'SDC-Payments'
GROUP  BY useridentity.arn
ORDER  BY scans DESC;

# Q3: Access from outside the expected VPC endpoint - exfiltration hunt
SELECT eventtime, useridentity.arn, eventname, sourceipaddress, vpcendpointid
FROM   cloudtrail_logs
WHERE  eventsource = 'dynamodb.amazonaws.com'
  AND  (vpcendpointid IS NULL OR vpcendpointid <> 'vpce-0abc123def456')
  AND  json_extract_scalar(requestparameters,'$.tableName') LIKE 'SDC-%'
ORDER  BY eventtime DESC;
```

Listing 10.1 — CloudTrail forensics with Athena. Query 3 is the one that finds the compromised credential.

10.2 Building an Immutable Audit Log with Streams

Streams' 24-hour retention makes it a transport, not an archive. The standard compliance pattern is Streams → Lambda (or Firehose) → S3 with Object Lock, giving you a WORM, queryable, permanent change history at a fraction of CloudTrail data-event cost.

MERMAID DIAGRAM — AUDIT PIPELINE

```
flowchart LR
    T[("DynamoDB Table")] -->|"NEW_AND_OLD_IMAGES"| S["DynamoDB Streams (24h)"]
    S --> L["Lambda: enrich + redact"]
    L --> F["Kinesis Data Firehose"]
    F -->|"Parquet, partitioned by date"| B["S3 Audit Bucket<br/>Object Lock = COMPLIANCE
mode<br/>7-year retention"]
    B --> G["Glue Catalog"]
    G --> A["Athena — auditor self-service"]
    L -->|"on failure"| D["SQS DLQ"]
    D --> AL["CloudWatch Alarm → on-call"]
    B --> Q["QuickSight — compliance dashboard"]
```

Figure 10.1 — Streams → Firehose → S3 Object Lock. Parquet + date partitioning keeps Athena costs trivial; COMPLIANCE-mode Object Lock means not even the root account can delete the evidence before the retention date.

PYTHON

```
# Lambda: turn a Stream record into an audit fact
import json, os, boto3
```

```
from datetime import datetime, timezone

firehose = boto3.client('firehose')
STREAM = os.environ['FIREHOSE_STREAM']
REDACT = {'ssn', 'card_number', 'password_hash'}

def redact(image: dict) -> dict:
    return {k: ('***REDACTED***' if k in REDACT else v) for k, v in (image or {}).items()}

def handler(event, context):
    records = []
    for r in event['Records']:
        d = r['dynamodb']
        # TTL deletions are attributed to the service, not a human
        principal = r.get('userIdentity', {}).get('principalId', 'application')
        fact = {
            'event_id':    r['eventID'],
            'event_name':  r['eventName'],              # INSERT | MODIFY | REMOVE
            'event_time':  datetime.fromtimestamp(
                               d['ApproximateCreationDateTime'],
                               tz=timezone.utc).isoformat(),
            'table_arn':   r['eventSourceARN'].split('/stream/')[0],
            'keys':        d['Keys'],
            'old_image':   redact(d.get('OldImage')),
            'new_image':   redact(d.get('NewImage')),
            'principal':   principal,
            'ttl_delete':  principal == 'dynamodb.amazonaws.com',
            'seq':         d['SequenceNumber'],
        }
        records.append({'Data': json.dumps(fact) + '\n'})

    for i in range(0, len(records), 500):          # Firehose caps at 500/batch
        firehose.put_record_batch(DeliveryStreamName=STREAM, Records=records[i:i+500])

    return {'audited': len(records)}
```

Listing 10.2 — The Stream-to-audit Lambda. Note the `ttl_delete` flag: `principalId == 'dynamodb.amazonaws.com'` is how you distinguish a TTL expiry from a real user deletion — auditors always ask.

WARNING — Lambda Stream consumer configuration is not optional

- `BisectBatchOnFunctionError: true` — a single poison record otherwise blocks the whole shard until the record expires.
- `MaximumRetryAttempts: 3` and `MaximumRecordAgeInSeconds: 3600` — bound the blast radius of a bad batch.
- `DestinationConfig.OnFailure` → an SQS DLQ, alarmed. Without it, failures are simply gone.
- `ParallelizationFactor: 1–10` — more concurrent consumers per shard **while still preserving per-key order**.
- Alarm on `IteratorAge > 4h`. At 24h, the records are deleted and the audit trail has a hole you cannot fill.

11. Backup & Recovery

PITR, on-demand backups, AWS Backup, S3 export — what each one protects against, and what it costs.

Table 11.1 — Backup options compared

Option	Granularity	Retention	Impact	Protects against
PITR	**Per second**, rolling	**35 days max**	Zero performance impact	Human error, bad deploy, logical corruption
On-demand backup	Point in time of the call	Indefinite	Zero; completes in seconds regardless of size	Pre-change snapshot, long-term retention
AWS Backup	Scheduled snapshots	Policy-driven (years)	Zero	Compliance, cross-account/Region, **Vault Lock ransomware defence**
S3 Export (PITR)	Any second in the PITR window	As long as the S3 object	**Zero table capacity consumed**	Analytics, migration, long-term archive
Global Tables	Continuous replica	Live	Replicated write cost	Regional outage — **NOT a backup** (a bad write replicates in <1s)

CRITICAL — Global Tables are not a backup

This confusion causes real data-loss incidents. A `DELETE` issued by a buggy deploy in us-east-1 arrives in eu-west-1 in under a second. Replication protects against **infrastructure** failure; PITR and backups protect against **logical** failure. You need both. If your DR plan says 'we have Global Tables so we don't need PITR', your DR plan is wrong.

11.1 Point-In-Time Recovery

BASH

```
# Enable (do this at table creation, not after the incident)
aws dynamodb update-continuous-backups --table-name SDC-Orders \
  --point-in-time-recovery-specification PointInTimeRecoveryEnabled=true

# What window do I actually have?
aws dynamodb describe-continuous-backups --table-name SDC-Orders \
  --query 'ContinuousBackupsDescription.PointInTimeRecoveryDescription'
# -> { "PointInTimeRecoveryStatus": "ENABLED",
#      "EarliestRestorableDateTime": "2026-06-10T...",
#      "LatestRestorableDateTime":   "2026-07-15T09:42:11Z" }   <- ~5 min behind now

# RESTORE - always to a NEW table. There is no in-place restore.
aws dynamodb restore-table-to-point-in-time \
  --source-table-name SDC-Orders \
  --target-table-name SDC-Orders-restore-20260715 \
  --restore-date-time 2026-07-15T08:15:00Z \
  --sse-specification-override 'Enabled=true,SSEType=KMS,KMSMasterKeyId=alias/sdc-ddb-cmk'
```

```
aws dynamodb wait table-exists --table-name SDC-Orders-restore-20260715

# Cross-Region / cross-account restore (PITR supports both)
aws dynamodb restore-table-to-point-in-time \
  --source-table-arn arn:aws:dynamodb:us-east-1:111122223333:table/SDC-Orders \
  --target-table-name SDC-Orders-dr \
  --restore-date-time 2026-07-15T08:15:00Z \
  --region eu-west-1
```

Listing 11.1 — PITR. `LatestRestorableDateTime` typically trails 'now' by about five minutes — that is your real RPO, not zero.

WARNING — What a PITR restore does NOT bring back

- **Auto Scaling policies** — the restored table has none. A restore into production traffic will throttle immediately.
- **IAM policies and resource policies** — nothing references the new table name.
- **Streams and its consumers** — the stream ARN is new; Lambda triggers must be re-created.
- **TTL configuration** — must be re-enabled.
- **Tags** — gone, so cost allocation breaks.
- **PITR itself** — the restored table starts with PITR *disabled*.
- **CloudWatch alarms** — they still point at the old table name.
- **Global Tables replication** — the restored table is single-Region.
- **Deletion protection** — off.
- **Restore duration** is proportional to table size and index count: minutes for GBs, **hours for TBs**. That is your RTO — measure it, do not assume it.

BASH

```
#!/usr/bin/env bash
# SDC-RUNBOOK-DDB-RESTORE-01 : restore + re-apply everything PITR drops
set -euo pipefail
SRC=SDC-Orders; TGT=SDC-Orders-restore-$(date +%Y%m%d%H%M); WHEN=$1

echo "[1/7] Restoring ${SRC} -> ${TGT} @ ${WHEN}"
aws dynamodb restore-table-to-point-in-time \
  --source-table-name "$SRC" --target-table-name "$TGT" --restore-date-time "$WHEN"
aws dynamodb wait table-exists --table-name "$TGT"

echo "[2/7] Re-enabling PITR"
aws dynamodb update-continuous-backups --table-name "$TGT" \
  --point-in-time-recovery-specification PointInTimeRecoveryEnabled=true

echo "[3/7] Re-enabling TTL"
aws dynamodb update-time-to-live --table-name "$TGT" \
  --time-to-live-specification 'Enabled=true,AttributeName=expires_at'

echo "[4/7] Re-enabling Streams"
aws dynamodb update-table --table-name "$TGT" \
  --stream-specification 'StreamEnabled=true,StreamViewType=NEW_AND_OLD_IMAGES'

echo "[5/7] Re-applying tags"
ARN=$(aws dynamodb describe-table --table-name "$TGT" --query 'Table.TableArn' --output text)
aws dynamodb tag-resource --resource-arn "$ARN" \
  --tags Key=Owner,Value=SQLDBAChamps Key=Env,Value=prod Key=CostCenter,Value=CC-4471

echo "[6/7] Deletion protection ON"
```

```
aws dynamodb update-table --table-name "$TGT" --deletion-protection-enabled

echo "[7/7] VALIDATE before cutover:"
aws dynamodb describe-table --table-name "$TGT" \
  --query
'Table.{Items:ItemCount,Bytes:TableSizeBytes,Status:TableStatus,GSIs:GlobalSecondaryIndexes[].IndexName}'
echo "NOTE: ItemCount updates only every ~6h and is APPROXIMATE. Spot-check known keys."
echo "Remaining MANUAL steps: Auto Scaling policies, Lambda stream trigger, CW alarms, app config
cutover."
```

Listing 11.2 — A restore runbook. The list of things you must re-apply is exactly why a PITR restore takes far longer than the restore call itself.

11.2 AWS Backup + Vault Lock — the ransomware control

BASH

```
# 1. Vault in a SEPARATE log/backup account, with its own CMK
aws backup create-backup-vault \
  --backup-vault-name sdc-ddb-vault \
  --encryption-key-arn arn:aws:kms:us-east-1:999988887777:key/abcd-...

# 2. Vault Lock in COMPLIANCE mode - AFTER the cooling-off window this is
#    IRREVERSIBLE. Not even AWS Support or the root user can delete a recovery
#    point before its retention date. That is the entire point.
aws backup put-backup-vault-lock-configuration \
  --backup-vault-name sdc-ddb-vault \
  --min-retention-days 35 \
  --max-retention-days 2555 \
  --changeable-for-days 3          # <-- cooling-off period; omit at your peril

# 3. Plan: daily 35d + monthly 7y, copied cross-Region and cross-account
aws backup create-backup-plan --backup-plan '{
  "BackupPlanName": "sdc-ddb-plan",
  "Rules": [
    { "RuleName": "daily-35d",
      "TargetBackupVaultName": "sdc-ddb-vault",
      "ScheduleExpression": "cron(0 5 ? * * *)",
      "StartWindowMinutes": 60, "CompletionWindowMinutes": 180,
      "Lifecycle": { "DeleteAfterDays": 35 },
      "CopyActions": [{
        "DestinationBackupVaultArn":
          "arn:aws:backup:eu-west-1:999988887777:backup-vault:sdc-ddb-vault-dr",
        "Lifecycle": { "DeleteAfterDays": 35 } }] },
    { "RuleName": "monthly-7y",
      "TargetBackupVaultName": "sdc-ddb-vault",
      "ScheduleExpression": "cron(0 6 1 * ? *)",
      "Lifecycle": { "DeleteAfterDays": 2555, "MoveToColdStorageAfterDays": 90 } }
  ]}'

# 4. Select tables by TAG, not by name - new tables are protected automatically
aws backup create-backup-selection \
  --backup-plan-id <plan-id> \
  --backup-selection '{
    "SelectionName": "all-prod-ddb",
    "IamRoleArn": "arn:aws:iam::111122223333:role/AWSBackupDefaultServiceRole",
    "ListOfTags": [{ "ConditionType": "STRINGEQUALS",
                     "ConditionKey": "Env", "ConditionValue": "prod" }]}'
```

Listing 11.3 — Tag-based selection means a table created next quarter is backed up on day one without anyone remembering. Vault Lock in COMPLIANCE mode is the control that survives a compromised admin.

11.3 S3 Export — backup for analytics

```bash
# Full export of any point in the PITR window - consumes ZERO table capacity
aws dynamodb export-table-to-point-in-time \
  --table-arn arn:aws:dynamodb:us-east-1:111122223333:table/SDC-Orders \
  --s3-bucket sdc-ddb-exports \
  --s3-prefix orders/2026-07-15/ \
  --export-time 2026-07-15T00:00:00Z \
  --export-format DYNAMODB_JSON

# INCREMENTAL export - only what changed between two times. Far cheaper for a
# nightly analytics feed than re-exporting the whole table.
aws dynamodb export-table-to-point-in-time \
  --table-arn arn:aws:dynamodb:us-east-1:111122223333:table/SDC-Orders \
  --s3-bucket sdc-ddb-exports --s3-prefix orders/incr/2026-07-15/ \
  --export-type INCREMENTAL_EXPORT \
  --incremental-export-specification \
     'ExportFromTime=2026-07-14T00:00:00Z,ExportToTime=2026-07-
15T00:00:00Z,ExportViewType=NEW_AND_OLD_IMAGES'

aws dynamodb list-exports --table-arn arn:aws:dynamodb:us-east-1:111122223333:table/SDC-Orders

# IMPORT from S3 into a NEW table (bulk load without burning WCU)
aws dynamodb import-table \
  --s3-bucket-source S3Bucket=sdc-ddb-exports,S3KeyPrefix=orders/2026-07-15/ \
  --input-format DYNAMODB_JSON \
  --table-creation-parameters '{
      "TableName": "SDC-Orders-clone",
      "AttributeDefinitions": [{"AttributeName":"PK","AttributeType":"S"},
                               {"AttributeName":"SK","AttributeType":"S"}],
      "KeySchema": [{"AttributeName":"PK","KeyType":"HASH"},
                    {"AttributeName":"SK","KeyType":"RANGE"}],
      "BillingMode": "PAY_PER_REQUEST"}'
```

Listing 11.4 — Export/Import. `ImportTable` is dramatically cheaper than `BatchWriteItem` for a bulk load and never throttles your live table.

INTERVIEW ANGLE — Interview angle — RPO/RTO

"What's your RPO and RTO for DynamoDB?" A weak answer says "PITR gives 5 minutes." A strong answer: *"RPO is ~5 minutes with PITR, or sub-second with Global Tables — but Global Tables replicate logical corruption, so for a bad-deploy scenario my RPO is the PITR window. RTO is dominated by restore time, which scales with table size and index count, plus the manual re-application of Auto Scaling, Streams, TTL, tags and alarms. I measure it in a quarterly game day rather than quoting a number from a slide."*

12. Disaster Recovery

Regional failure, logical corruption, ransomware, and accidental deletion — four different disasters that need four different answers.

12.1 Match the Strategy to the Disaster

Table 12.1 — Disaster taxonomy

Disaster	Primary control	RPO	RTO
AZ failure	Built-in 3-AZ replication	**Zero**	**Zero** — invisible to you
Region failure	Global Tables (active-active)	< 1 s (typical)	Seconds to minutes (DNS/config cutover)
Region failure (no Global Tables)	PITR cross-Region restore	≈ 5 min	Hours (restore + re-config)
Logical corruption (bad deploy)	PITR restore to just before	≈ 5 min	Restore time + validation
Accidental `DeleteTable`	Deletion protection; then PITR/backup	≈ 5 min	Restore time
Ransomware / malicious insider	AWS Backup Vault Lock, cross-account	Last backup	Restore time
Account compromise	Cross-account backup copy in an isolated account	Last copy	Hours to days
KMS key disabled/deleted	SCP protection + key recovery	N/A	Minutes (re-enable) or **total loss**

MERMAID DIAGRAM — MULTI-REGION DR ARCHITECTURE

```
flowchart TB
    subgraph R53["Route 53 — health-check based failover / latency routing"]
        DNS["api.sqldbachamps.com"]
    end
    subgraph P["us-east-1 (Primary)"]
        A1["ALB → ECS/Lambda"]
        T1[("SDC-Orders replica")]
        B1["PITR 35d + AWS Backup vault"]
        A1 --> T1
        T1 --> B1
    end
    subgraph S["eu-west-1 (Secondary — ACTIVE)"]
        A2["ALB → ECS/Lambda"]
        T2[("SDC-Orders replica")]
        B2["PITR 35d + backup copy"]
        A2 --> T2
        T2 --> B2
    end
    subgraph LOG["999988887777 — isolated backup account"]
        V["Vault Lock — COMPLIANCE mode<br/>7-year retention"]
    end
    DNS -->|"health check"| A1
    DNS -->|"failover / latency"| A2
    T1 <-->|"Global Tables — bidirectional, ~sub-second, LWW"| T2
    B1 -.->|"cross-account copy"| V
```

```
    B2 -.-> V
    style V fill:#FDEDEC,stroke:#C0392B
```

Figure 12.1 — Global Tables handle infrastructure failure; the locked vault in a separate account handles the human and the attacker. Both are required.

12.2 Global Tables — the operational reality

```bash
# Prerequisite: Streams with NEW_AND_OLD_IMAGES
aws dynamodb update-table --table-name SDC-Orders \
  --stream-specification 'StreamEnabled=true,StreamViewType=NEW_AND_OLD_IMAGES'

# Add a replica Region (Version 2019.11.21)
aws dynamodb update-table --table-name SDC-Orders \
  --replica-updates '[{"Create": {
        "RegionName": "eu-west-1",
        "KMSMasterKeyId": "alias/sdc-ddb-cmk-eu",
        "PointInTimeRecovery": {"PointInTimeRecoveryEnabled": true}}}]'

# Monitor the backfill (can take hours on a large table; the table stays online)
watch -n 30 "aws dynamodb describe-table --table-name SDC-Orders \
  --query 'Table.Replicas[].{Region:RegionName,Status:ReplicaStatus}' --output table"

# Verify replication lag - THE metric for Global Tables health
aws cloudwatch get-metric-statistics \
  --namespace AWS/DynamoDB --metric-name ReplicationLatency \
  --dimensions Name=TableName,Value=SDC-Orders Name=ReceivingRegion,Value=eu-west-1 \
  --start-time "$(date -u -d '1 hour ago' +%Y-%m-%dT%H:%M:%SZ)" \
  --end-time "$(date -u +%Y-%m-%dT%H:%M:%SZ)" \
  --period 300 --statistics Average Maximum
```

Listing 12.1 — Adding a replica is online but the backfill is not instant. Add replicas before you need them, never during an incident.

CRITICAL — Last-writer-wins in production terms

Two users update the same order in two Regions within the replication window. LWW picks the higher internal timestamp; **the other write is silently discarded — no error, no conflict record, no metric.** Mitigations: (1) route each entity to a home Region (e.g. by customer ID hash) so concurrent cross-Region writes cannot happen; (2) model as append-only events instead of in-place updates; (3) use conditional writes with a version attribute so at least the loser gets an error locally; (4) for workloads that genuinely cannot tolerate LWW, evaluate the multi-Region strong consistency option and accept its latency and cost.

12.3 DR Runbook Template

Table 12.2 — SDC-RUNBOOK-DDB-DR-01: regional failover

#	Step	Command / action	Owner & target
1	Declare the incident; open the bridge	PagerDuty → #sdc-incident	On-call DBA / T+0
2	Confirm it is regional, not application	AWS Health Dashboard + `describe-table` in both Regions	DBA / T+2 min
3	Check replication lag before cutover	`AgeOfOldestUnreplicatedRecord` in eu-west-1	DBA / T+3 min

#	Step	Command / action	Owner & target
4	**Decision:** lag > 5 min?	If yes, quantify data loss with the business **before** cutting over	Incident Commander / T+5 min
5	Fail over traffic	Route 53 health check flips, or manually set the failover record	SRE / T+7 min
6	Pre-scale the secondary	Secondary WCU/RCU must handle **100%** of global traffic	SRE / T+8 min
7	Verify writes land	Canary write + strongly consistent read in eu-west-1	DBA / T+10 min
8	Communicate	Status page + stakeholder update	IC / T+15 min
9	Monitor for replication catch-up	When us-east-1 returns, expect a backlog flush and LWW conflicts	DBA / ongoing
10	Fail back only when stable	Do **not** fail back during the recovery flush	IC / T+24 h
11	Post-incident review	Blameless PIR; update this runbook	All / T+5 days

WARNING — The capacity trap in every DR plan

If your secondary Region is provisioned for its normal 20% share of traffic, failover sends it 100% and it throttles instantly — turning a Region failure into a total outage. **Either provision every replica for full global load (expensive but honest), or run replicas in On-Demand (which absorbs up to 2× previous peak — still not 5×), or pre-scale as step 6 of the runbook and accept a few minutes of degradation.** Choose deliberately and write the choice down.

12.4 Game Days

- **Quarterly, in production or a production-shaped clone.** A DR plan that has never been executed is a document, not a capability.
- **Measure, don't assume:** actual restore time for your real table size, actual failover time, actual data loss.
- **Test the ugly path:** restore into a table with no Auto Scaling and watch it throttle. That is what will happen at 3 a.m.
- **Rotate the operator.** If only one person can run the runbook, you do not have a runbook.
- **Test the ransomware scenario:** can you restore from the locked vault in the isolated account with your primary account's admin credentials revoked?

13. High Availability

Most DynamoDB HA is free and automatic. The interesting part is the HA you must design yourself.

13.1 What AWS Gives You for Free

Table 13.1 — Built-in availability

Mechanism	Detail
3-AZ synchronous replication	Every partition has three replicas in three AZs. Writes ACK at 2/3 quorum.
Automatic leader election	Paxos-based. An AZ loss triggers a new leader in seconds, transparently.
Stateless request routers	A fleet, load-balanced, self-healing. No connection to fail over.
Automatic partition healing	A failed storage node is replaced and re-replicated with no human action.
Single-Region SLA	**99.99%** — about 52 minutes of allowed downtime per year.
Global Tables SLA	**99.999%** — about 5 minutes per year.

> **NOTE — The contrast that matters to a SQL Server DBA**
> There is **no** Always On AG to configure, no WSFC quorum to model, no listener, no synchronous-commit latency penalty to negotiate, no failover partner, no patching order and no manual failover test. AZ-level HA is not a feature you enable — it is the product. Your entire HA effort goes into the layers *above* the database: correct client retry behaviour, capacity headroom, and multi-Region if you need five nines.

13.2 HA Comparison

Table 13.2 — SQL Server Always On AG vs. DynamoDB

Dimension	SQL Server Always On AG	DynamoDB
Setup effort	Days: WSFC, quorum, listener, certs, endpoints	**Zero** — it is the default
Failover time	Seconds to minutes; app must reconnect	Seconds; **transparent** to the client
Sync commit cost	Latency penalty on every commit	Built into the 2/3 quorum — no toggle
Manual failover test	Required, disruptive, scheduled	N/A — AWS handles it continuously
Read scale-out	Readable secondaries	Any replica serves eventual reads automatically
Multi-Region	Distributed AG — complex	**Global Tables — one API call**
Split-brain risk	Real; needs careful quorum design	Managed internally by Paxos
Patching downtime	Rolling upgrades, planned windows	**None — no maintenance windows at all**
Your responsibility	Nearly everything	Retry logic, capacity headroom, key design

13.3 The HA You Do Have To Build

Table 13.3 — Availability failures that are yours, not AWS's

Failure mode	Why it takes you down	Your control
Throttling	The service is up; **your** requests are rejected	Correct key design + headroom + adaptive retries
No retry logic	A transient throttle becomes a user-visible 500	SDK `retryMode=adaptive`, `maxAttempts≥5`
Retry storm	Aggressive retries amplify a small throttle into an outage	Exponential backoff **with full jitter**; circuit breaker
Hot partition	One key throttles while the table is 5% utilised	Contributor Insights + write sharding
Auto Scaling lag	A 2–7 min gap during a spike	Scheduled pre-scaling or On-Demand
`UnprocessedItems` ignored	Silent data loss with an HTTP 200	Always loop until the map is empty
Single-Region design	Region outage = full outage	Global Tables + a tested failover
DAX node failure	Cache stampede onto the table	≥3 nodes across AZs; the table must survive a 100% miss rate
CMK disabled	Table becomes unreadable in minutes	SCP + alarm on `DisableKey`

```python
# Exponential backoff with FULL JITTER - the AWS-recommended algorithm.
# Equal jitter and no jitter both produce retry convoys that keep the table hot.
import random, time
from botocore.exceptions import ClientError

RETRYABLE = {
    'ProvisionedThroughputExceededException',
    'ThrottlingException',
    'RequestLimitExceeded',
    'TransactionConflictException',
    'InternalServerError',
    'ServiceUnavailable',
}

def with_backoff(fn, *args, max_attempts=8, base=0.05, cap=20.0, **kwargs):
    for attempt in range(max_attempts):
        try:
            return fn(*args, **kwargs)
        except ClientError as e:
            code = e.response['Error']['Code']
            if code not in RETRYABLE or attempt == max_attempts - 1:
                raise                    # 4xx client errors must NOT be retried
            # Full jitter: sleep = uniform(0, min(cap, base * 2**attempt))
            time.sleep(random.uniform(0, min(cap, base * (2 ** attempt))))
    raise RuntimeError('unreachable')
```

Listing 13.1 — Full-jitter backoff. Retrying a `ValidationException` or `ConditionalCheckFailedException` is a bug: they will never succeed, and you have just doubled your load.

INTERVIEW ANGLE — Interview angle

> "How do you achieve high availability with DynamoDB?" The trap is to describe AWS's architecture back to the interviewer. The senior answer flips it: *"Single-Region HA is free — three AZs, quorum writes, automatic leader election, 99.99% SLA, and there's nothing for me to configure. So the real HA work is on my side of the API: adaptive retries with full jitter, a partition key that doesn't create a hot spot, capacity headroom or On-Demand, and Global Tables if the business needs five nines. In my experience, DynamoDB outages are almost always throttling caused by key design, not AWS failing."*

14. Troubleshooting

A decision tree, the five incidents you will actually see, and the commands that resolve them.

MERMAID DIAGRAM — MASTER TROUBLESHOOTING DECISION TREE

```
flowchart TD
    S["Problem reported"] --> Q1{"Errors or latency?"}

    Q1 -->|"Errors"| E1{"HTTP status?"}
    E1 -->|"400"| E2{"Error code?"}
    E2 -->|"ProvisionedThroughputExceeded"| T1["THROTTLING — go to A"]
    E2 -->|"ValidationException"| T2["Malformed request — check<br/>key types, reserved
words,<br/>attribute definitions"]
    E2 -->|"ConditionalCheckFailed"| T3["Working as designed —<br/>optimistic lock lost"]
    E2 -->|"ResourceNotFound"| T4["Wrong table / wrong Region<br/>/ wrong account"]
    E2 -->|"AccessDenied"| T5["IAM — simulate-principal-policy;<br/>check SCP + endpoint policy +
KMS key policy"]
    E1 -->|"5xx"| T6["Retry with backoff;<br/>check AWS Health Dashboard"]

    Q1 -->|"Latency"| L1{"Server-side<br/>SuccessfulRequestLatency high?"}
    L1 -->|"No — server is fast"| L2["CLIENT-SIDE: pool exhaustion, DNS,<br/>cold start, GC, no
keep-alive.<br/>Check RetryAttempts — hidden throttling."]
    L1 -->|"Yes"| L3{"Which operation?"}
    L3 -->|"Scan"| L4["Delete the Scan.<br/>Add a GSI or use S3 Export"]
    L3 -->|"Query"| L5{"1 MB page limit hit?"}
    L5 -->|"Yes"| L6["Paginate; narrow the<br/>key condition; project less"]
    L5 -->|"No"| L7["Item too large — compress<br/>or move blobs to S3"]
    L3 -->|"GetItem"| L8["Large item (4 KB rounding)<br/>or a strong read on a hot leader"]

    T1 --> A0{"A: which dimension<br/>is throttling?"}
    A0 -->|"GlobalSecondaryIndexName"| A6["GSI backpressure stalls BASE TABLE writes.<br/>Raise
GSI capacity; register Auto Scaling<br/>on dynamodb:index:* — routinely forgotten.<br/>Then ask:
is this GSI read at all?"]
    A0 -->|"TableName"| A1{"Consumed ≈ Provisioned?"}
    A1 -->|"Yes"| A2["Genuine under-provisioning:<br/>raise capacity / fix Auto
Scaling<br/>min+max / consider On-Demand"]
    A1 -->|"No — table is 5% used!"| A3{"HOT PARTITION.<br/>Which key?"}
    A3 --> A4["Contributor Insights →<br/>MostThrottledKeys"]
    A4 --> A5["Write sharding, higher-cardinality PK,<br/>DAX for reads, or redesign.<br/>Do NOT
raise capacity — it dilutes throughput."]
```

Figure 14.1 — Start here. The highest-value branch is 'which dimension is throttling?': a GSI-dimension throttle stalls base-table writes and is invisible on table-level metrics. The second is 'Consumed ≈ Provisioned?' — if no, you have a hot partition, and adding capacity makes it permanently worse.

14.1 Incident 1 — Throttling at 5% Table Utilisation

Table 14.1 — Symptom → diagnosis → fix

Stage	What you see	What you do
Symptom	`ProvisionedThroughputExceededException` while `ConsumedWCU` is 500 of 10,000	Do **not** raise capacity — you will dilute throughput permanently
Hypothesis	Traffic is concentrated on one partition key	Confirm before acting
Diagnose	Contributor Insights → `MostThrottledKeys`	Names the exact key
Confirm	One key = 90% of writes (e.g. `PK='2026-07-15'` or `PK='PENDING'`)	Now you know the design flaw

Stage	What you see	What you do
Fix — tactical	Write sharding: `PK = '2026-07-15#' + shard(0..9)`	Buys 10× write headroom
Fix — reads	DAX in front of the hot read key	Microsecond reads; the hot spot never reaches the table
Fix — strategic	Re-key on a high-cardinality attribute	The only permanent answer

14.2 Incident 2 — GSI Throttling Stalls the Base Table

This one baffles people because the base table looks healthy. A GSI has its own capacity; when a GSI throttles, DynamoDB applies backpressure and **the write to the base table fails too**. Your orders stop being accepted because an index you barely use is under-provisioned.

```bash
# 1. Is it the table, or an index? The dimension you must add is IndexName.
for IDX in GSI1 GSI2 GSI3; do
  echo "--- $IDX ---"
  aws cloudwatch get-metric-statistics \
    --namespace AWS/DynamoDB --metric-name WriteThrottleEvents \
    --dimensions Name=TableName,Value=SDC-Orders Name=GlobalSecondaryIndexName,Value=$IDX \
    --start-time "$(date -u -d '30 minutes ago' +%Y-%m-%dT%H:%M:%SZ)" \
    --end-time "$(date -u +%Y-%m-%dT%H:%M:%SZ)" \
    --period 60 --statistics Sum --output text
done

# 2. Fix immediately: raise the GSI's own capacity
aws dynamodb update-table --table-name SDC-Orders \
  --global-secondary-index-updates '[{"Update": {
     "IndexName": "GSI1",
     "ProvisionedThroughput": {"ReadCapacityUnits": 500, "WriteCapacityUnits": 500}}}]'

# 3. Prevent recurrence: register Auto Scaling for the INDEX dimension
aws application-autoscaling register-scalable-target \
  --service-namespace dynamodb \
  --resource-id "table/SDC-Orders/index/GSI1" \
  --scalable-dimension "dynamodb:index:WriteCapacityUnits" \
  --min-capacity 100 --max-capacity 4000

# 4. Ask the real question: is this GSI used at all?
aws cloudwatch get-metric-statistics \
  --namespace AWS/DynamoDB --metric-name ConsumedReadCapacityUnits \
  --dimensions Name=TableName,Value=SDC-Orders Name=GlobalSecondaryIndexName,Value=GSI3 \
  --start-time "$(date -u -d '30 days ago' +%Y-%m-%dT%H:%M:%SZ)" \
  --end-time "$(date -u +%Y-%m-%dT%H:%M:%SZ)" --period 86400 --statistics Sum
# Zero reads for 30 days + full write cost on every write = delete it.
```

Listing 14.1 — GSI throttling. Step 4 is the one that saves money: unused GSIs are pure cost and pure risk.

14.3 Incident 3 — 'It's Slow' but the Server Is Fast

Table 14.2 — When SuccessfulRequestLatency is 3 ms and users see 400 ms

Client-side cause	Signature	Fix
Connection pool exhaustion	Latency grows with concurrency; server latency flat	`max_pool_connections` ≥ thread count

Client-side cause	Signature	Fix
No TCP keep-alive	Every request pays a TLS handshake (~20–40 ms)	Enable keep-alive (SDK v3 / boto3 default)
Lambda cold start	P99 spikes after idle; P50 fine	Provisioned concurrency; init the SDK client outside the handler
Client in the wrong Region	+70–150 ms constant	Co-locate, or add a Global Tables replica
No VPC endpoint (NAT hop)	Extra latency + NAT data charges	Gateway endpoint
Retries hidden inside the SDK	Server sees 3 ms; client sees 400 ms after 3 silent retries	Log `RetryAttempts` from the response metadata
DNS resolution per request	Sawtooth latency	Reuse the SDK client — never construct one per request
GC / thread starvation	Correlates with app CPU, not DynamoDB	Profile the app; DynamoDB is innocent

PYTHON
```
# Prove where the time goes: measure server-side vs. wall-clock
import time, boto3

ddb = boto3.client('dynamodb')   # created ONCE, module scope - never per request

t0 = time.perf_counter()
resp = ddb.get_item(
    TableName='SDC-Orders',
    Key={'PK': {'S': 'ORDER#9001'}, 'SK': {'S': 'META'}},
    ReturnConsumedCapacity='TOTAL'
)
wall_ms = (time.perf_counter() - t0) * 1000

md = resp['ResponseMetadata']
print(f"wall            : {wall_ms:.1f} ms")
print(f"retries         : {md.get('RetryAttempts')}")     # >0 means hidden throttling
print(f"http status     : {md['HTTPStatusCode']}")
print(f"consumed RCU    : {resp['ConsumedCapacity']['CapacityUnits']}")
print(f"request id      : {md['RequestId']}")             # give this to AWS Support

# wall >> SuccessfulRequestLatency (CloudWatch) => the problem is on YOUR side.
# RetryAttempts > 0 => you ARE being throttled; the SDK is hiding it from you.
```

Listing 14.2 — `RetryAttempts` is the field that ends the 'DynamoDB is slow' argument. A non-zero value means the SDK silently absorbed throttling.

14.4 Incident 4 — TTL Items Never Expire

BASH
```
# Step 1: is TTL even enabled and on the right attribute?
aws dynamodb describe-time-to-live --table-name Sessions
# Look for: {"TimeToLiveStatus": "ENABLED", "AttributeName": "expires_at"}
# ENABLING / DISABLING can take up to ~1 hour to settle.

# Step 2: inspect a real item - this is where the bug always is
aws dynamodb get-item --table-name Sessions \
  --key '{"session_id": {"S": "abc-123"}}' \
  --query 'Item.expires_at'
```

```
#  {"N": "1784000000"}      OK - epoch SECONDS, type N
#  {"S": "1784000000"}      BROKEN - it is a String. TTL ignores it. Silently. Forever.
#  {"N": "1784000000000"}   BROKEN - milliseconds -> expires in the year 58,500
#  {"N": "1784000000.5"}    BROKEN - non-integer
#  attribute absent          Item never expires (this is legal and often intentional)
#  wrong attribute NAME      TTL config says 'ttl', item has 'expires_at' -> nothing happens

# Step 3: is TTL actually deleting anything?
aws cloudwatch get-metric-statistics \
  --namespace AWS/DynamoDB --metric-name TimeToLiveDeletedItemCount \
  --dimensions Name=TableName,Value=Sessions \
  --start-time "$(date -u -d '7 days ago' +%Y-%m-%dT%H:%M:%SZ)" \
  --end-time "$(date -u +%Y-%m-%dT%H:%M:%SZ)" --period 86400 --statistics Sum
# Flat zero for 7 days while items are expiring = configuration bug, not a delay.
```

Listing 14.3 — The TTL failure mode is silence. There is no error, no metric spike and no warning — just a storage bill that never stops growing.

> **NOTE — TTL expectations**
>
> TTL deletes typically within 48 hours of expiry — it is a background sweeper, not a cron job. If your application requires items to be *gone* at the second, filter on `expires_at > :now` in the read path and treat TTL purely as storage reclamation.

14.5 Incident 5 — Stream Consumer Lag Approaching 24 Hours

BASH

```
# THE CLOCK IS RUNNING. At 24h the records are gone permanently.

# 1. How bad?  (IteratorAge in ms; 86,400,000 = 24h = data loss)
aws cloudwatch get-metric-statistics \
  --namespace AWS/Lambda --metric-name IteratorAge \
  --dimensions Name=FunctionName,Value=sdc-orders-stream-consumer \
  --start-time "$(date -u -d '3 hours ago' +%Y-%m-%dT%H:%M:%SZ)" \
  --end-time "$(date -u +%Y-%m-%dT%H:%M:%SZ)" --period 300 --statistics Maximum

# 2. Is a poison pill blocking a shard? Errors flat-lining with age climbing = yes.
aws logs filter-log-events \
  --log-group-name /aws/lambda/sdc-orders-stream-consumer \
  --start-time $(( ($(date +%s) - 3600) * 1000 )) \
  --filter-pattern 'ERROR'

# 3. Immediate mitigation: bisect on error + cap retries + send failures to a DLQ.
#    Without these, ONE bad record blocks its shard for the full 24 hours.
aws lambda update-event-source-mapping \
  --uuid <esm-uuid> \
  --bisect-batch-on-function-error \
  --maximum-retry-attempts 3 \
  --maximum-record-age-in-seconds 3600 \
  --parallelization-factor 10 \
  --destination-config '{"OnFailure": {"Destination":
      "arn:aws:sqs:us-east-1:111122223333:sdc-stream-dlq"}}'

# 4. Throughput mitigation: raise ParallelizationFactor (per-key order is preserved)
#    and raise the function's reserved concurrency.

# 5. If records WILL expire: reconcile from the source of truth, do not pretend.
#    Run an incremental S3 export over the gap window and replay it.
aws dynamodb export-table-to-point-in-time \
```

```
  --table-arn arn:aws:dynamodb:us-east-1:111122223333:table/SDC-Orders \
  --s3-bucket sdc-ddb-exports --s3-prefix gap-replay/ \
  --export-type INCREMENTAL_EXPORT \
  --incremental-export-specification \
     'ExportFromTime=2026-07-14T02:00:00Z,ExportToTime=2026-07-
14T06:00:00Z,ExportViewType=NEW_AND_OLD_IMAGES'
```

Listing 14.4 — Step 5 is the one people forget: PITR incremental export can reconstruct the change set that the Stream dropped, provided you are still inside the 35-day PITR window.

14.6 Triage Command Cheat Sheet

BASH

```
T=SDC-Orders

# --- Table shape and health ---
aws dynamodb describe-table --table-name $T \
  --query
'Table.{Status:TableStatus,Items:ItemCount,GB:TableSizeBytes,Mode:BillingModeSummary.BillingMode,RCU:Prov
isionedThroughput.ReadCapacityUnits,WCU:ProvisionedThroughput.WriteCapacityUnits,GSIs:GlobalSecondaryInde
xes[].{Name:IndexName,Status:IndexStatus},Stream:StreamSpecification,Protected:DeletionProtectionEnabled}
'


# --- Backup posture ---
aws dynamodb describe-continuous-backups --table-name $T
aws dynamodb list-backups --table-name $T --query
'BackupSummaries[].{Name:BackupName,When:BackupCreationDateTime,Size:BackupSizeBytes}'

# --- Throttling in the last hour (table level) ---
aws cloudwatch get-metric-statistics --namespace AWS/DynamoDB \
  --metric-name ThrottledRequests --dimensions Name=TableName,Value=$T \
  --start-time "$(date -u -d '1 hour ago' +%Y-%m-%dT%H:%M:%SZ)" \
  --end-time "$(date -u +%Y-%m-%dT%H:%M:%SZ)" --period 300 --statistics Sum \
  --query 'sort_by(Datapoints,&Timestamp)[].{T:Timestamp,Throttles:Sum}' --output table

# --- Consumed vs provisioned ---
aws cloudwatch get-metric-statistics --namespace AWS/DynamoDB \
  --metric-name ConsumedWriteCapacityUnits --dimensions Name=TableName,Value=$T \
  --start-time "$(date -u -d '1 hour ago' +%Y-%m-%dT%H:%M:%SZ)" \
  --end-time "$(date -u +%Y-%m-%dT%H:%M:%SZ)" --period 60 --statistics Sum Maximum

# --- Cost of a single query (the ReturnConsumedCapacity trick) ---
aws dynamodb query --table-name $T \
  --key-condition-expression 'PK = :p' \
  --expression-attribute-values '{":p": {"S": "CUST#1001"}}' \
  --return-consumed-capacity INDEXES \
  --query 'ConsumedCapacity'

# --- Can this role actually do that? ---
aws iam simulate-principal-policy \
  --policy-source-arn arn:aws:iam::111122223333:role/sdc-app-role \
  --action-names dynamodb:PutItem dynamodb:Scan dynamodb:DeleteTable \
  --resource-arns arn:aws:dynamodb:us-east-1:111122223333:table/$T \
  --query 'EvaluationResults[].{Action:EvalActionName,Decision:EvalDecision}' --output table
```

Listing 14.5 — The commands to run in the first five minutes of any DynamoDB incident. `--return-consumed-capacity INDEXES` is the fastest way to see what a query really costs.

15. Common Errors

Every error code you will meet, what actually causes it, and whether to retry.

Table 15.1 — Error reference

Error code	HTTP	Retry?	Real cause and fix
`ProvisionedThroughputExceededException`	400	**Yes** (backoff)	Capacity exceeded — usually a **hot partition**, not the table. Check consumed vs. provisioned first. Fix: shard the key, raise capacity, or move to On-Demand
`ThrottlingException`	400	**Yes**	Control-plane rate limit (too many `UpdateTable`/`CreateTable` calls). Slow your IaC down
`RequestLimitExceeded`	400	**Yes**	Account-level throughput quota reached. Raise the quota via Service Quotas
`ValidationException`	400	**No**	Malformed request. Top causes: key type mismatch (`S` vs `N`), an undeclared key attribute, `ConsistentRead=true` on a GSI, an empty key, expression >4 KB
`ConditionalCheckFailedException`	400	**No**	**Working as designed.** Your `ConditionExpression` was false. Handle it as business logic, never retry blindly
`TransactionCanceledException`	400	Depends	Read `CancellationReasons[]`. `ConditionalCheckFailed` → no. `TransactionConflict`/`ThrottlingError` → yes, with backoff
`TransactionConflictException`	400	**Yes**	Another transaction touched the same item. Contention hot spot — reduce it
`ItemCollectionSizeLimitExceededException`	400	**No**	**LSI 10 GB cliff.** A partition key's collection exceeded 10 GB. Requires a table redesign without the LSI
`ResourceNotFoundException`	400	**No**	Wrong table name, wrong Region, wrong account — or the table is still `CREATING`
`ResourceInUseException`	400	**No**	Table is `CREATING`/`UPDATING`/`DELETING`. `wait table-exists` before acting
`LimitExceededException`	400	**No**	Too many concurrent control-plane ops, or too many tables/GSIs
`AccessDeniedException`	400	**No**	IAM. Check in order: identity policy → **SCP** → VPC endpoint policy → resource policy → **KMS key policy**. The KMS one is the most-missed
`UnrecognizedClientException`	400	**No**	Bad/expired credentials, or clock skew >5 min (SigV4 fails). Check NTP
`RequestExpired`	400	**No**	Clock skew. Fix the system clock

Error code	HTTP	Retry?	Real cause and fix
`IdempotentParameterMismatchException`	400	**No**	Reused a `ClientRequestToken` with different parameters
`InternalServerError`	500	**Yes**	AWS-side. Retry with backoff; check the Health Dashboard if sustained
`ServiceUnavailable`	503	**Yes**	AWS-side. Retry with backoff

15.1 The Reserved-Word Trap

DynamoDB has **over 570 reserved words**, including `name`, `status`, `size`, `timestamp`, `date`, `year`, `count`, `data`, `value`, `type`, `source`, `owner`, `region`, `hash`, `key`, `state`, `user`, `group` and `comment`. Using any of them unescaped in an expression throws `ValidationException`, and the error message is unhelpfully generic.

```python
# BROKEN - 'status' and 'timestamp' are reserved words
table.update_item(
    Key={'PK': 'ORDER#1', 'SK': 'META'},
    UpdateExpression='SET status = :s, timestamp = :t',
    ExpressionAttributeValues={':s': 'SHIPPED', ':t': '2026-07-15'}
)
# botocore.exceptions.ClientError: ValidationException:
#   Invalid UpdateExpression: Attribute name is a reserved keyword; keyword: status

# CORRECT - alias EVERY attribute name with #, always. Make it a habit, not a fix.
table.update_item(
    Key={'PK': 'ORDER#1', 'SK': 'META'},
    UpdateExpression='SET #st = :s, #ts = :t',
    ExpressionAttributeNames={'#st': 'status', '#ts': 'timestamp'},
    ExpressionAttributeValues={':s': 'SHIPPED', ':t': '2026-07-15'}
)
```

Listing 15.1 — The rule that eliminates the whole class of bug: alias every attribute name in every expression, whether or not you think it is reserved.

15.2 The `TransactionCanceledException` Puzzle

```python
# TransactWriteItems fails with ONE opaque error for up to 100 items.
# The detail is in CancellationReasons - and it is positional.
from botocore.exceptions import ClientError

try:
    client.transact_write_items(TransactItems=[
        {'Update': {'TableName': 'SDC-Accounts',
                    'Key': {'PK': {'S': 'ACCT#A'}},
                    'UpdateExpression': 'SET bal = bal - :amt',
                    'ConditionExpression': 'bal >= :amt',
                    'ExpressionAttributeValues': {':amt': {'N': '100'}}}},
        {'Update': {'TableName': 'SDC-Accounts',
                    'Key': {'PK': {'S': 'ACCT#B'}},
                    'UpdateExpression': 'SET bal = bal + :amt',
                    'ExpressionAttributeValues': {':amt': {'N': '100'}}}},
    ], ClientRequestToken='transfer-2026-07-15-0001')   # idempotency: 10-min window

except ClientError as e:
    if e.response['Error']['Code'] != 'TransactionCanceledException':
        raise
```

```
    # CancellationReasons[i] maps 1:1 to TransactItems[i]
    for i, r in enumerate(e.response['CancellationReasons']):
        code = r.get('Code')
        if code == 'None':
            continue                      # this item was fine
        print(f'item[{i}] failed: {code} - {r.get("Message")}')
        # ConditionalCheckFailed -> business error (insufficient funds). DO NOT RETRY.
        # TransactionConflict    -> contention. RETRY with backoff.
        # ThrottlingError        -> capacity.  RETRY with backoff.
        # ValidationError        -> your bug.  DO NOT RETRY.

# ClientRequestToken makes the transaction idempotent for 10 minutes -
# essential when a network timeout leaves you unsure whether it committed.
```

Listing 15.2 — Never log `TransactionCanceledException` without unpacking `CancellationReasons`. The top-level message tells you nothing.

15.3 Silent Failures — the dangerous category

Table 15.2 — Bugs that never raise an error

Silent failure	Consequence	Detection
Ignoring `UnprocessedItems`	**Permanent data loss** behind an HTTP 200	Assert the returned map is empty; alarm on it
Not paginating `LastEvaluatedKey`	Only the first 1 MB is processed; reports quietly wrong	Assert `LastEvaluatedKey` is absent, or always loop
TTL attribute as `S` or in ms	Items never expire; storage grows forever	Alarm if `TimeToLiveDeletedItemCount` = 0
Global Tables LWW conflict	**A write vanishes with no error at all**	Application-level versioning + reconciliation job
DAX stale after a direct write	Reads return old data for up to the item TTL	Route every writer through DAX
Float precision in `N`	`0.1 + 0.2 != 0.3`; money is wrong	Use `Decimal` (Python) / `BigDecimal` (Java)
`Scan` behind PartiQL `SELECT`	SQL-looking code hides a full table scan	Ban PartiQL in application code; review `ConsumedCapacity`

```python
# THE PAGINATION BUG - a 'complete' report that quietly stops at 1 MB

# WRONG: silently returns only the first page (max 1 MB)
items = table.query(KeyConditionExpression=Key('PK').eq('CUST#1001'))['Items']

# RIGHT: paginate until LastEvaluatedKey is gone
def query_all(table, **kwargs):
    items, start = [], None
    while True:
        if start:
            kwargs['ExclusiveStartKey'] = start
        resp = table.query(**kwargs)
        items.extend(resp['Items'])
        start = resp.get('LastEvaluatedKey')
        if not start:
```

```
            return items

# BEST: use the paginator and never think about it again
paginator = boto3.client('dynamodb').get_paginator('query')
for page in paginator.paginate(TableName='SDC-Orders',
                               KeyConditionExpression='PK = :p',
                               ExpressionAttributeValues={':p': {'S': 'CUST#1001'}}):
    process(page['Items'])

# Note: Limit=N does NOT mean 'return N items'. It means 'evaluate at most N items'.
# With a FilterExpression you can get ZERO items back AND a LastEvaluatedKey.
# Code that treats an empty page as 'end of results' is broken.
```

Listing 15.3 — Pagination. The `Limit` semantics in the closing comment catch even experienced engineers.

16. FAQs

The questions that come up on every project, answered directly.

Q1. Can I change a table's partition key after creation?

No. The key schema is immutable. You must create a new table and migrate — typically via an S3 export, a transform, and `ImportTable`, or a dual-write plus Streams backfill for a zero-downtime cutover. This is precisely why access-pattern enumeration is a prerequisite, not a nice-to-have.

Q2. Can I add or remove an LSI later?

No for both. LSIs exist only from `CreateTable` onward and can never be dropped. GSIs can be added and removed online at any time. This asymmetry alone is why the default answer is 'use a GSI'.

Q3. How do I do a `COUNT(*)`?

There is no O(1) count. `describe-table` returns `ItemCount`, but it is **approximate and refreshed roughly every six hours** — never use it for correctness. Options: (a) `Select=COUNT` on a Query/Scan (still bills for everything read); (b) maintain a counter item updated by a Streams-driven Lambda using an atomic `ADD`; (c) export to S3 and count in Athena. Option (b) is the standard production answer.

Q4. Why am I throttled when the table is only 5% utilised?

A hot partition. Capacity is allocated **per partition** (1,000 WCU / 3,000 RCU each), and one partition key cannot exceed that no matter what the table total is. Use Contributor Insights → `MostThrottledKeys` to name the key, then shard it or re-key. Raising table capacity does not help and permanently dilutes throughput across more partitions.

Q5. On-Demand or Provisioned?

Start On-Demand. After two to four weeks of CloudWatch data, switch to Provisioned + Auto Scaling + Reserved Capacity **if** utilisation is steady and above roughly 20%. On-Demand costs ~6–7× per request but you pay only for what you use; the break-even is around 14–18% average utilisation. Also: set an On-Demand maximum throughput cap so a runaway loop cannot generate a five-figure bill overnight.

Q6. Are Global Tables a backup?

No, and believing they are causes data loss. Replication copies your mistakes in under a second. Global Tables protect against a *Region* failing; PITR and AWS Backup protect against a *deploy* or a *person*. You need both, and you should be able to say that sentence in an incident review.

Q7. Can I query a GSI with strong consistency?

No. `ConsistentRead=true` against a GSI throws `ValidationException`. GSI propagation is typically sub-second but is not bounded. If you need read-after-write on an index, either restructure so the read hits the base table, or accept the window and design for it (e.g. return the just-written item from the write response rather than re-reading).

Q8. How do I store something bigger than 400 KB?

Put the object in S3 and store the key in DynamoDB. If it is a large document that must stay logically atomic, split it across multiple items in one item collection (same partition key) and reassemble on read. The 400 KB limit is hard and non-negotiable — do not design around it with compression alone.

Q9. Single-table design — is it mandatory?

No. It is a technique, not a religion. It minimises round trips and cost and is excellent when access patterns are stable. It also hurts readability, complicates analytics handoff and raises the onboarding cost for new engineers. Choose it

deliberately: single-table for stable, latency-critical, high-scale domains; multi-table when the domain is still evolving or the team is new to DynamoDB. Being unable to argue both sides is the actual red flag.

Q10. Does `ProjectionExpression` save me money?

It saves **network bandwidth and latency**, not RCU. Capacity is charged on the size of the item as stored, before projection is applied. To reduce read *cost*, reduce the stored item size, or read from a GSI with a narrower projection.

Q11. Is PartiQL a good idea?

For the console and ad-hoc exploration, yes. **In application code, no.** PartiQL wraps the same API in SQL syntax, so a `SELECT` without a full key condition silently becomes a `Scan`. It gives you SQL's ergonomics with none of SQL's optimiser, which is the worst of both. Use the native API where the cost of every call is visible.

Q12. How do I implement a unique constraint on a non-key attribute (e.g. email)?

Create a second item whose partition key *is* the email (`PK='EMAIL#a@b.com'`), and write both items in a single `TransactWriteItems` with `ConditionExpression='attribute_not_exists(PK)'` on the email item. The transaction fails if the email exists. This is the canonical DynamoDB unique-constraint pattern, and it is a common interview question.

Q13. How do I paginate for a UI?

Return the `LastEvaluatedKey` to the client as an opaque, **signed or encrypted** cursor and accept it back as `ExclusiveStartKey`. Never expose it raw — it contains real key values and lets a client jump to arbitrary items. There is no `OFFSET`; you cannot jump to page 47. If the UI needs numbered pages, DynamoDB is the wrong store for that view.

Q14. Does DynamoDB support server-side triggers?

Not in the SQL Server sense. Streams + Lambda is the equivalent: asynchronous, at-least-once-ish (effectively exactly-once per record with Lambda), 24-hour retention, ordered per partition key. It runs **after** the write commits, so it cannot veto a write — use a `ConditionExpression` for that.

Q15. What is the fastest way to bulk-load 500 GB?

`ImportTable` from S3 into a **new** table. It is far cheaper than `BatchWriteItem`, does not consume table WCU, and cannot throttle your live traffic. If you must load into an existing table, use On-Demand, parallel `BatchWriteItem` with full-jitter backoff, and expect throughput dilution afterwards if you use Provisioned.

Q16. Can I run analytics on DynamoDB?

Not directly, and you should not try. Use `ExportTableToPointInTime` to S3 (zero table capacity consumed) and query with Athena, or stream to Redshift/OpenSearch via Kinesis. Running a `Scan` for a report is the most common way teams take down their own production table.

Q17. How much does an empty table cost?

In On-Demand, effectively nothing (only storage). In Provisioned, you pay for every provisioned RCU/WCU-hour whether you use them or not — so an idle Provisioned table with 1,000 RCU/1,000 WCU costs roughly $600/month for nothing. PITR adds ~$0.20/GB-month on top. Idle Provisioned tables are the most common piece of DynamoDB waste in any account.

Q18. How do I delete all items in a table?

Delete the table and recreate it. It is instant and free. Scanning and deleting 100 million items costs 100 million WCU and hours of wall time. If the table must survive (because of an ARN referenced elsewhere), use TTL to expire everything — TTL deletes are free.

Q19. Is DynamoDB ACID?

Per item, yes: every single-item write is atomic and durable. Across items, only inside `TransactWriteItems` (up to 100 items / 4 MB, at 2× WCU, with serializable isolation). There are no long-running transactions, no `BEGIN`/`COMMIT`, no locks you hold, and no isolation level to choose.

Q20. When is DAX genuinely worth it?

When reads are repetitive, dominate your bill, and can tolerate up-to-5-minute staleness — and when every writer for that table can go through DAX. If any writer bypasses it, you have a coherence bug rather than a cache. If your problem is a *write* hot spot, DAX does nothing at all.

17. Interview Questions

Graded from screening to architect, with the answers that distinguish a senior candidate.

17.1 Level 1 — Screening

Q1. What type of database is DynamoDB?

Fully managed, serverless, key-value **and** document NoSQL. Regional service, multi-AZ by default, single-digit-millisecond latency, schemaless apart from the primary key.

Q2. Explain partition key and sort key.

The partition key is hashed to decide **physical placement** — which partition holds the item. The sort key orders items **within** that partition and enables range queries (`begins_with`, `BETWEEN`, `>`, `<`). Together they form the primary key and must be unique. A `Query` always requires an equality on the partition key.

Q3. Query vs. Scan?

`Query` reads only items under one partition key and bills for what it returns. `Scan` reads every item in the table and bills for all of them — a `FilterExpression` is applied afterwards and saves nothing. `Scan` in an OLTP path is almost always a design bug.

Q4. Eventually vs. strongly consistent reads?

Eventually consistent reads may be served by a replica that has not yet applied the newest write (typically <1s of staleness) and cost half an RCU per 4 KB. Strongly consistent reads go to the partition leader, always reflect all prior successful writes, cost a full RCU, and are **not available on GSIs**.

Q5. What is the maximum item size?

400 KB, including attribute names. It is a hard limit. For larger payloads, store the object in S3 and keep the pointer in DynamoDB.

17.2 Level 2 — Mid-Level

Q6. GSI vs. LSI — and which do you default to?

GSI: any partition/sort key, created or dropped online at any time, separate capacity, eventually consistent only, 20 per table. LSI: must share the base partition key, **create-time only and undroppable**, shares base capacity, supports strong reads, 5 per table, and imposes a hard **10 GB item-collection limit per partition key**.

I default to GSIs. The only thing an LSI adds is strongly consistent index reads, and that is rarely worth an irreversible design decision plus a 10 GB cliff that can hard-fail writes in production.

Q7. Calculate RCU/WCU: 500 reads/sec of 6 KB items, eventually consistent.

`ceil(6 / 4) = 2` RCU for a strong read → `2 / 2 = 1` RCU eventually consistent → `500 × 1 = 500 RCU`.

The follow-up to volunteer: writes round on **1 KB**, so 500 writes/sec of the same 6 KB item would be `500 × 6 = 3,000 WCU` — six times the cost, which is why write-heavy designs live or die on item size.

Q8. You're throttled but the table shows 5% utilisation. What's happening?

A hot partition. Capacity is per-partition (1,000 WCU / 3,000 RCU), not per-table, and a single partition key is indivisible. Diagnose with Contributor Insights → `MostThrottledKeys`. Fix with write sharding, a higher-cardinality key, or DAX for read hot spots. Critically: **do not raise table capacity** — it does not help and permanently splits the table, diluting throughput after you scale back down.

Q9. What happens to `UnprocessedItems`?

`BatchWriteItem` returns HTTP 200 with an `UnprocessedItems` map when some writes were throttled. **The SDK does not retry them for you.** You must loop until the map is empty, with exponential backoff and full jitter. Ignoring it is silent, permanent data loss — and it is the single most common DynamoDB bug I see in code review.

Q10. How do you implement optimistic concurrency?

A `version` attribute plus `ConditionExpression='#v = :expected'` and `SET #v = #v + 1` in the same `UpdateItem`. On `ConditionalCheckFailedException`, re-read and retry the business operation. This gives compare-and-swap semantics at 1× WCU, versus 2× for a transaction.

17.3 Level 3 — Senior

Q11. Design a table for a multi-tenant SaaS with 10,000 tenants, where one tenant is 40% of traffic.

The naive `PK = tenant_id` gives a permanent hot partition for the whale tenant, and no capacity increase fixes it.

Approach: `PK = tenant_id#entity_id` for high cardinality within each tenant, with `dynamodb:LeadingKeys` restricted to `${tenant_id}` for row-level isolation. For the whale, add write sharding on its hot entities. If the whale genuinely exceeds one table's practical envelope, give it a **dedicated table** — the noisy-neighbour and blast-radius argument usually wins with the business anyway, and it lets you bill it separately.

Trade-off to state out loud: per-tenant tables give perfect isolation but 10,000 tables hits the soft quota and makes deployment painful. A hybrid — shared table for the long tail, dedicated tables for the top handful — is what I'd actually ship.

Q12. Walk me through your DR strategy.

Four different disasters need four different controls. AZ failure: free and automatic. Region failure: Global Tables, with **every replica provisioned for full global load** or on On-Demand — otherwise failover throttles the survivor and turns a Region outage into a total outage. Logical corruption: PITR (~5 min RPO), because Global Tables replicate the bad write in under a second. Ransomware or a compromised admin: AWS Backup with **Vault Lock in COMPLIANCE mode** in a separate account, so not even root can delete the recovery points.

RTO is dominated by restore time plus re-applying everything PITR drops — Auto Scaling, Streams triggers, TTL, tags, alarms, PITR itself. I measure that in a quarterly game day rather than quoting a slide.

Q13. How would you audit every read of a PII attribute for a PCI-DSS assessor?

Two layers. (1) **CloudTrail data events scoped to that table only** — account-wide data events are financially ruinous on a busy table. (2) **Streams → Lambda → Firehose → S3 with Object Lock in COMPLIANCE mode**, giving a WORM, Athena-queryable change history for the seven-year retention.

Then the preventive controls the assessor will actually score: PAN encrypted **client-side** with the AWS Database Encryption SDK so the service never sees plaintext; `dynamodb:Attributes` + `Select=SPECIFIC_ATTRIBUTES` for column-level restriction; `aws:SecureTransport` enforced; a CMK with an SCP denying `kms:ScheduleKeyDeletion`. Server-side encryption alone does **not** satisfy Req. 3 against an over-permissioned application role, and saying so is usually what convinces the assessor you understand the control.

Q14. A GSI is throttling. What breaks, and why does it surprise people?

GSI throttling **backpressures writes to the base table** — your `PutItem` calls fail even though the base table has plenty of capacity. People are surprised because they only monitor table-level metrics; you must add the `GlobalSecondaryIndexName` dimension to see it.

Fix: raise the GSI's own capacity, register Auto Scaling for the `dynamodb:index:*` dimensions (routinely forgotten), or put the GSI on On-Demand. Then ask whether the GSI is used at all — an index with zero reads in 30 days is paying full write cost on every single write, and deleting it is both a cost win and an availability win.

Q15. When would you tell a stakeholder that DynamoDB is the wrong choice?

When the access patterns aren't known or won't stabilise; when the workload is ad-hoc analytics or BI; when items exceed 400 KB; when the domain needs real joins and referential integrity at low latency; when the team needs `GROUP BY`; or when volume is low enough that Aurora Serverless is cheaper and vastly more flexible.

The framing I'd use: *DynamoDB trades query flexibility for predictable scale. If you can't tell me your access patterns today, you're buying the cost without the benefit, and you'll pay for it in a migration in eighteen months.*

17.4 Level 4 — Architect / Scenario

Q16. Design an e-commerce order system: 50M customers, 500M orders/year, sub-10 ms P99, active-active US/EU.

Model (single-table): `PK=CUST#<id>`, `SK=PROFILE | ORDER#<ts>#<id> | ADDR#<id>` — one Query returns a customer with their newest orders, no joins. Order line items live in the order's own item collection (`PK=ORDER#<id>`).

Indexes: GSI1 sparse on `status=PENDING` for the fulfilment queue (index size tracks the backlog, not the table); GSI2 `STORE#<id>` + date range for store reporting; GSI3 on the tracking number, `KEYS_ONLY`.

Capacity: On-Demand with a maximum throughput cap — Black Friday is precisely the spike Auto Scaling handles badly.

Multi-Region: Global Tables US/EU, but **route each customer to a home Region by ID hash** so concurrent cross-Region writes to one order cannot happen. That sidesteps last-writer-wins entirely rather than hoping it never bites.

Analytics: nightly incremental S3 export → Athena. Zero table capacity, no Scan, no risk to the OLTP path.

Ops: PITR + Vault Lock in a separate account; Streams → OpenSearch for order search; alarms on ThrottledRequests, IteratorAge and AgeOfOldestUnreplicatedRecord.

The thing I'd flag to the business: 500M orders/year with GSIs is a large bill; I'd model the cost of GSI2 and GSI3 explicitly before committing, because a reporting index that nobody reads is a 25% tax on every write.

Q17. Migrate a 2 TB SQL Server table to DynamoDB with zero downtime.

Phase 0 — design. Enumerate every access pattern from the existing query log (`sys.dm_exec_query_stats` and Query Store give you this for free — most teams skip it and guess). Anything that is a join or an aggregate does **not** move; it goes to S3/Athena.

Phase 1 — dual write. The application writes to both stores, with SQL Server remaining the source of truth. Log and alarm on any DynamoDB write failure.

Phase 2 — backfill. SQL Server → S3 (Parquet) via bcp or DMS, transform, then `ImportTable` into a **new** DynamoDB table. Never backfill with `BatchWriteItem` — `ImportTable` is cheaper and cannot throttle live traffic.

Phase 3 — reconcile. A validation job compares row counts and a sampled checksum on both sides. Do not proceed until it is clean for several days.

Phase 4 — shadow reads. Read from DynamoDB, compare against SQL Server, log divergences. Serve from SQL Server.

Phase 5 — cut over reads, then writes, keeping SQL Server as a fallback for a defined period.

Phase 6 — decommission.

The honest caveat: if Phase 0 reveals patterns that DynamoDB can't serve, the correct outcome of this project is to *not migrate*, and I'd say that in the kickoff meeting rather than in month nine.

Q18. Your DynamoDB bill tripled last month. Walk me through the investigation.

1. Cost Explorer by usage type — split WriteCapacityUnit-Hrs, ReadCapacityUnit-Hrs, TimedStorage, Streams, PITR, and data transfer. This alone usually names the culprit.

2. The four usual suspects, in order: a new GSI (every write now costs 2×); a `Scan` someone added to a report; an idle Provisioned table nobody scaled down; PITR enabled on a huge table nobody restores from.

3. `ConsumedWriteCapacityUnits` per index — if GSI writes climbed but reads didn't, you're paying for an index nobody queries.

4. `ReturnedItemCount` vs. `ConsumedReadCapacityUnits` — a huge gap is the signature of a filtered Scan.

5. On-Demand with no max throughput cap — check for a retry loop; a bug that retries forever bills forever.

6. Structural fixes: delete unused GSIs, `INCLUDE` instead of `ALL`, compress large attributes, Standard-IA for cold tables, Reserved Capacity for the steady baseline, and a max-throughput cap so this cannot recur silently.

> **INTERVIEW ANGLE — The meta-skill interviewers are actually testing**
> Notice that the strongest answers above all share a shape: **state the trade-off, name the failure mode, then give the recommendation.** Candidates who recite feature lists sound like documentation. Candidates who say *"I'd default to X, because Y is irreversible and fails at 3 a.m. in this specific way"* sound like someone who has been on-call. Every answer in this section is built to that pattern — use it.

18. Real-World Use Cases

Six patterns, with the key design that makes each one work.

18.1 Session Store

Table 18.1 — Session store design

Aspect	Design
Why DynamoDB	Pure key-value; millions of tiny items; TTL removes the cleanup job entirely
Key schema	`PK = session_id` (UUID — perfect cardinality). No sort key
TTL	`expires_at` = epoch seconds. **Free** deletes; no cron, no cleanup Lambda
Capacity	On-Demand — traffic follows human activity, which is spiky
Consistency	Eventually consistent reads; a session is not a bank balance
Correctness note	Still filter `expires_at > :now` on read — TTL deletes within ~48h, not at the second
Anti-pattern avoided	`PK = user_id` would concentrate a power user's sessions on one partition

18.2 IoT Time-Series

```text
REQUIREMENT : 1,000,000 devices x 1 reading/minute = ~16,700 writes/sec

✗ PK = 'readings'            -> 1 partition, 1,000 WCU max. Dead on arrival.
✗ PK = timestamp_bucket      -> today's bucket is hot. Dead by lunchtime.
☑ PK = device_id             -> 1M keys, uniform. ~0.017 WCU per key.
   SK = ISO-8601 timestamp   -> range queries per device come free.

Item : { PK: 'DEV#a1b2c3',
         SK: '2026-07-15T09:30:00Z',
         t: 21.4, h: 55, batt: 87,        <- short names: WCU is charged on total size
         expires_at: 1786592400 }         <- TTL: 90-day raw retention

ACCESS PATTERNS
  Latest reading for a device  : Query PK=DEV#x, ScanIndexForward=false, Limit=1
  Readings in a window         : Query PK=DEV#x, SK BETWEEN t1 AND t2
  Fleet-wide aggregate         : NOT DynamoDB -> Streams -> Firehose -> S3 -> Athena

TIERING
  Hot  (0-90d)   : DynamoDB Standard, TTL to expire
  Warm (90d-1y)  : Streams -> S3 Parquet (Athena)   [set up BEFORE the TTL fires]
  Cold (1y+)     : S3 Glacier

WHY THIS WORKS: the write hot spot is impossible because 1M device IDs spread
perfectly, and TTL keeps the table small forever without a single delete costing WCU.
```

Listing 18.1 — The time-series pattern. Note that the fleet-wide aggregate deliberately leaves DynamoDB: forcing it to stay would mean a Scan.

18.3 Shopping Cart (the original Dynamo use case)

- **Key:** `PK = CART#<customer_id>`, `SK = ITEM#<sku>` — one Query returns the whole cart in a single round trip.
- **Atomic quantity update:** `UpdateExpression='ADD qty :n'` — no read-modify-write race, no lost update.

- **Abandoned carts:** TTL at 30 days. Free, automatic, no batch job.
- **Checkout:** `TransactWriteItems` to create the order, decrement inventory with `ConditionExpression='stock >= :n'`, and clear the cart — all-or-nothing.
- **Why not relational:** availability. The 2007 Dynamo paper existed because 'add to cart' must never fail, even during a partition. That requirement has not changed.

18.4 Gaming Leaderboard

Table 18.2 — Leaderboard design and its honest limitation

Aspect	Design
Base table	`PK = GAME#<id>#<season>`, `SK = PLAYER#<id>` — player's own score, O(1)
Top-N GSI	`GSI1PK = GAME#<id>#<season>`, `GSI1SK = score` (N) → Query descending, `Limit=100`
Hot partition risk	**Real** — one game+season is one partition key. Mitigate with DAX (reads are 99% of traffic) and/or shard the GSI PK into buckets, merging on read
Score update	`UpdateItem` with `ADD score :points` — atomic, no read first
Limitation to state	There is **no efficient way to get a player's rank** ('you are #4,127'). It would require counting everyone above them. Real systems maintain rank in ElastiCache/Redis (`ZRANK`) or approximate it from score buckets. Saying this out loud in an interview is a strong signal
Caching	DAX in front — leaderboards are read-mostly and tolerate seconds of staleness

18.5 Financial Ledger (compliance-heavy)

MERMAID DIAGRAM — LEDGER WITH AN IMMUTABLE AUDIT TRAIL

```
flowchart TB
    API["Payments API"]

    subgraph TX["One TransactWriteItems — all four commit, or none do"]
        X1["Debit ACCT#A — ConditionExpression: bal >= amt"]
        X2["Credit ACCT#B"]
        X3["Append EVENT#uuid — append-only, never updated"]
        X4["Idempotency item — attribute_not_exists(PK)"]
    end

    API --> TX
    TX -->|"atomic commit — 2× WCU"| T[("SDC-Ledger")]

    T -->|"NEW_AND_OLD_IMAGES"| S["Streams"]
    S --> L["Lambda — redact PAN, enrich"]
    L --> FH["Firehose (Parquet)"]
    FH --> S3["S3 — Object Lock COMPLIANCE, 7 years"]
    S3 --> ATH["Athena — auditor self-service"]

    T -.->|"continuous, 35-day window"| PITR["PITR"]
    T -.->|"scheduled snapshot + cross-account copy"| BK["AWS Backup → Vault Lock<br/>separate account"]

    style S3 fill:#FDEDEC,stroke:#C0392B
    style BK fill:#FDEDEC,stroke:#C0392B
```

Figure 18.1 — The transaction is the input to the ledger, not an output of it. Append-only events, an idempotency item inside the same transaction, and a WORM audit trail that survives a compromised admin.

- **Never store a running balance only.** Store append-only events plus a materialised balance item, updated in the same transaction. The events are the truth; the balance is a cache you can rebuild.
- **Idempotency is not optional.** A network timeout leaves the client unsure whether the transfer committed. Use `ClientRequestToken` (10-minute window) **and** an idempotency item with `attribute_not_exists(PK)` for anything longer.
- **PAN/PII encrypted client-side** with the AWS Database Encryption SDK — server-side encryption does not protect against an over-permissioned role.
- **Never active-active across Regions for a ledger** without a conflict strategy: last-writer-wins on a balance is how money silently disappears. Pin each account to a home Region.
- **Deny `dynamodb:DeleteTable` and `dynamodb:DeleteBackup`** to every role except break-glass, and alarm on both.

18.6 Event Sourcing / CQRS

Table 18.3 — Event store design

Aspect	Design
Key schema	`PK = AGG#<aggregate_id>`, `SK = <sequence_number>` (zero-padded: `0000000042`)
Append	`PutItem` with `ConditionExpression='attribute_not_exists(SK)'` — optimistic concurrency on the sequence; a duplicate sequence is rejected, never overwritten
Replay	`Query PK=AGG#x` returns every event in order, in one call
Snapshots	Every N events, write `SK='SNAPSHOT#<n>'` so replay doesn't start from zero
Read models	Streams → Lambda → projections into other DynamoDB tables, OpenSearch or Redshift
Immutability	Events are **never updated or deleted**. Deny `UpdateItem`/`DeleteItem` on the events table in IAM — let the policy enforce the architecture
Why it fits	Append-only + key-ordered reads is exactly what DynamoDB's sort key does best, and unbounded event volume is exactly where relational event stores start hurting

19. Best Practices

The consolidated rules, ordered by how much pain they save.

19.1 Data Modelling

1. **Enumerate every access pattern before creating the table.** This is not advice; it is the prerequisite. A missed pattern is a migration, not an index.
2. **Choose the highest-cardinality partition key that serves your patterns.** `user_id` good; `status` catastrophic.
3. **Design so that `Query` serves every OLTP read.** If a pattern needs a `Scan`, the model is wrong.
4. **Default to GSIs.** LSIs are create-time-only, undroppable, and carry a hard 10 GB item-collection cliff.
5. **Use sparse GSIs for queues and filtered views** — the index costs scale with the subset, not the table.
6. **Use `INCLUDE`, not `ALL`, for GSI projections.** `ALL` doubles storage and write cost, often to project attributes nobody reads.
7. **Keep items small.** Short attribute names, compress big blobs, and move anything over ~100 KB to S3. Reads round up to 4 KB; writes round up to 1 KB.
8. **Use composite sort keys** (`ORDER#2026-07-15#9001`) to encode hierarchy and get range queries for free.
9. **Overload GSIs** (generic `GSI1PK`/`GSI1SK`) so one index serves several entity types — you only get 20.
10. **Decide single-table vs. multi-table deliberately**, and be able to argue both sides.

19.2 Operations

11. **Enable PITR and deletion protection on every production table, at creation time.** Both are off by default; both are free-ish insurance against the incident you have not had yet.
12. **Start On-Demand; move to Provisioned + Auto Scaling + Reserved Capacity only with data.**
13. **Set an On-Demand maximum throughput cap.** It is your only defence against a retry loop billing five figures overnight.
14. **Register Auto Scaling for GSI dimensions too.** The single most common capacity gap in production accounts.
15. **Never `Scan` in production.** Use `ExportTableToPointInTime` → Athena: zero table capacity, zero risk.
16. **Alarm on `ThrottledRequests`, `SystemErrors`, `IteratorAge` and `AgeOfOldestUnreplicatedRecord`** before the table takes traffic.
17. **Manage everything as code.** Click-ops on a table you cannot re-key later is how tribal knowledge becomes an outage.
18. **Tag every table** (Owner, Env, CostCenter, DataClassification) — it drives cost allocation, ABAC and tag-based backup selection.
19. **Review GSI usage quarterly.** A GSI with zero reads in 30 days is a pure tax on every write.
20. **Run a DR game day quarterly** and measure the real restore time, not the one on the slide.

19.3 Application Code

21. **Always handle `UnprocessedItems` and `LastEvaluatedKey`.** These two omissions cause silent data loss and silently wrong reports respectively.
22. **Use `retryMode=adaptive` with full-jitter backoff.** Never retry `ValidationException` or `ConditionalCheckFailedException`.

23. **Create the SDK client once**, at module scope, with keep-alive and a pool sized to your concurrency.
24. **Alias every attribute name with `#`.** 570+ reserved words; the habit costs nothing.
25. **Use `ConditionExpression` for every write that has an invariant.** It is compare-and-swap at 1× WCU.
26. **Use `ClientRequestToken` on transactions** for idempotency across network timeouts.
27. **Use `Decimal`/`BigDecimal`, never float**, for anything that represents money.
28. **Prefer one `Query` over N `GetItem` calls** — rounding happens once per request, not once per item.
29. **Ban PartiQL in application code.** SQL syntax over a key-value store hides Scans behind a familiar keyword.
30. **Log `RetryAttempts` and `ConsumedCapacity`** on your hot paths. It ends most 'DynamoDB is slow' arguments in one line.

19.4 Security

31. **Roles, never IAM users.** No long-lived access keys anywhere near a table.
32. **Explicitly deny `Scan`, `DeleteTable`, `DeleteBackup` and `ExportTableToPointInTime`** to application roles — the export API is a quiet exfiltration path.
33. **Customer-managed KMS key** on every regulated table; protect it with an SCP denying `ScheduleKeyDeletion`/`DisableKey`.
34. **VPC Gateway Endpoint with a restrictive endpoint policy** — it is a data-exfiltration control, not just a cost saving.
35. **`dynamodb:LeadingKeys` for multi-tenant isolation**; `dynamodb:Attributes` + `Select=SPECIFIC_ATTRIBUTES` for column-level control.
36. **Client-side encryption for PAN/PII.** Server-side encryption does not protect against an over-permissioned role.
37. **CloudTrail data events, scoped to regulated tables only** — account-wide is financially ruinous.
38. **Streams → S3 with Object Lock (COMPLIANCE)** for the permanent audit trail Streams' 24 hours cannot provide.
39. **AWS Backup Vault Lock in a separate account.** This is the control that survives a compromised admin.
40. **Never put PII in a partition key or a GSI key** — keys appear in logs, in `LastEvaluatedKey` cursors, and in CloudTrail.

> **TIP — The ten-second version**
>
> **Know your access patterns. Pick a high-cardinality partition key. Never Scan. Default to GSIs. Enable PITR and deletion protection. Handle `UnprocessedItems`. Alarm on throttling. Global Tables are not a backup.** If a team does only these eight things, they will avoid most of the incidents in this document.

20. References

Primary sources first. Where AWS documentation and a blog post disagree, the documentation wins.

20.1 AWS Primary Documentation

Table 20.1 — Official sources

Resource	URL
DynamoDB Developer Guide	https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/
Best Practices for Design & Architecture	https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/best-practices.html
Service Quotas & Limits	https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/ServiceQuotas.html
Reserved Words (all 570+)	https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/ReservedWords.html
API Reference	https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/
Error Handling	https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Programming.Errors.html
CloudWatch Metrics Reference	https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/metrics-dimensions.html
Global Tables	https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/GlobalTables.html
Point-in-Time Recovery	https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/PointInTimeRecovery.html
DynamoDB Streams	https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Streams.html
DAX Developer Guide	https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/DAX.html
DynamoDB Local	https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/DynamoDBLocal.html
AWS Database Encryption SDK	https://docs.aws.amazon.com/database-encryption-sdk/latest/devguide/
AWS Backup Vault Lock	https://docs.aws.amazon.com/aws-backup/latest/devguide/vault-lock.html
Pricing	https://aws.amazon.com/dynamodb/pricing/
Service Level Agreement	https://aws.amazon.com/dynamodb/sla/

20.2 Foundational Reading

Table 20.2 — Papers, tools and books

Resource	Detail / URL
Dynamo: Amazon's Highly Available Key-value Store (SOSP 2007)	The origin paper. Consistent hashing, quorums, vector clocks. https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf
Amazon DynamoDB: A Scalable, Predictably Performant, and Fully Managed NoSQL Database Service (USENIX ATC 2022)	How the *service* differs from the paper. Essential and under-read. https://www.usenix.org/conference/atc22/presentation/elhemali
NoSQL Workbench for DynamoDB	Model, visualise and generate code before you create the table. https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/workbench.html
AWS Well-Architected — Reliability & Cost Pillars	https://aws.amazon.com/architecture/well-architected/
The DynamoDB Book — Alex DeBrie	The definitive practitioner text on single-table design. https://www.dynamodbbook.com/
Alex DeBrie's blog	https://www.alexdebrie.com/
AWS Database Blog — DynamoDB category	https://aws.amazon.com/blogs/database/category/database/amazon-dynamodb/
re:Invent — 'Advanced Design Patterns for DynamoDB' (DAT403)	The annual session; search YouTube for the latest year

20.3 Certification Alignment

Table 20.3 — Where this material maps

Exam	DynamoDB coverage
AWS Certified Cloud Practitioner (CLF-C02)	Sections 1, 3 — what it is, when to use it
AWS Certified Solutions Architect – Associate (SAA-C03)	Sections 1, 2, 3, 11, 12, 13 — capacity modes, GSI vs. LSI, Global Tables, DAX
AWS Certified Database – Specialty (DBS-C01)	**All sections.** Especially 8 (tuning), 11–12 (backup/DR), 14–15 (troubleshooting)
AWS Certified Solutions Architect – Professional (SAP-C02)	Sections 2, 7, 12, 13, 18 — multi-Region, security, migration patterns
AWS Certified Developer – Associate (DVA-C02)	Sections 3, 8, 15 — expressions, error handling, batch and transaction APIs

NOTE — About this document

Published by DB Learning Series.

Prices, quotas and service behaviours cited here reflect publicly documented AWS behaviour and are indicative only — **always verify current quotas and pricing against the AWS documentation and the Pricing Calculator for your Region before making a design or budget decision.**

Corrections and contributions are welcome.